Univeristà degli Studi di Milano-Bicocca
Dipartimento di Matematica

Matematica Numerica per il Machine Learning

Relazione di Laboratorio

Autore Luca Lucentini
Matricola 916891

Settembre 2025
A.A. 2024/2025

Introduzione

La presente relazione riporta il lavoro svolto durante le sessioni di laboratorio del corso di Matematica Numerica per il Machine Learning nell'anno accademico 2024/2025. L'obiettivo delle attività è stato quello di introdurre e sperimentare in modo pratico le principali architetture di reti neurali artificiali, analizzandone i meccanismi di base e implementandone esempi concreti in Python. In particolare, sono state considerate le reti neurali feed-forward e le reti convoluzionali, discutendo il loro funzionamento, il meccanismo di addestramento, le tecniche di ottimizzazione e il calcolo dei gradienti tramite backpropagation. Gli esperimenti proposti hanno permesso di osservare direttamente le capacità di classificazione di questi modelli, sia in contesti semplici (dataset sintetici) sia in applicazioni più realistiche (come la classificazione di immagini).

1 Reti neurali feed-forward

Le reti neurali feed-forward sono una delle architetture fondamentali del deep learning. L'unità elementare di una rete neurale è il *neurone*.

Definizione 1 (Neurone)**.** Dato un vettore di input $\mathbf{x} \in \mathbb{R}^m$, un neurone è una funzione $f : \mathbb{R}^m \to \mathbb{R}$, definita da

$$f(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b),$$

dove $\mathbf{w} \in \mathbb{R}^m$ è il vettore dei pesi, $b \in \mathbb{R}$ è il bias, e $\sigma : \mathbb{R} \to \mathbb{R}$ è una funzione di attivazione, tipicamente non lineare. Di seguito elenchiamo alcune delle scelte più comuni per l'attivazione:

- **Sigmoide**: $\sigma(x) = \dfrac{1}{1+e^{-x}}, \quad x \in \mathbb{R}$;
- **ReLU**: $\mathrm{ReLU}(x) = \max(0, x), \quad x \in \mathbb{R}$;
- **Tangente iperbolica**: $\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}, \quad x \in \mathbb{R}$;
- **Softmax**: $\mathrm{softmax}(\mathbf{x})_i = \dfrac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}, \quad \mathbf{x} \in \mathbb{R}^m, \quad i = 1, \ldots, m.$

Un neurone rappresenta dunque una trasformazione che combina linearmente gli ingressi e introduce non linearità attraverso la funzione di attivazione. A questo punto, un singolo neurone può già essere interpretato come un modello supervisionato; se fossimo in presenza di un dataset di N punti $(\mathbf{x}_j, y_j)_{j=1}^N$, dove le $\mathbf{x}_j \in \mathbb{R}^m$ rappresentano le features e le $y_j \in \mathbb{R}$ le labels, potremmo definire una funzione costo e minimizzarla rispetto ai parametri $(\mathbf{w}, b)$, ottenendo un modello addestrato. Tuttavia, la capacità predittiva di un singolo neurone è limitata: per costruire modelli più espressivi, i neuroni vengono organizzati in parallelo a formare un *layer*. Un layer è costituito da un insieme di neuroni che ricevono lo stesso vettore di input e producono in uscita un nuovo vettore, che diventa l'ingresso del layer successivo. Più formalmente, consideriamo un layer con n neuroni che riceve in ingresso un vettore $\mathbf{x} \in \mathbb{R}^m$. Siano $\mathbf{w}_i, b_i$ i pesi e il bias del neurone i-esimo, allora il layer è la funzione $h : \mathbb{R}^m \to \mathbb{R}^n$, definita da

$$h(\mathbf{x}) = \sigma(W\mathbf{x} + \mathbf{b}),$$

dove $W \in \mathbb{R}^{n \times m}$ e $\mathbf{b} \in \mathbb{R}^n$, definiti da

$$W = \begin{bmatrix} \mathbf{w}_1^\top \\ \vdots \\ \mathbf{w}_n^\top \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix},$$

sono rispettivamente la matrice dei pesi e il vettore dei bias, e σ viene applicata componente per componente. A partire dalla definizione di layer, una rete neurale feed-forward si ottiene disponendo più layer in sequenza, come si può vedere in Figura 1. Generalmente una rete feed-forward ha la seguente struttura:

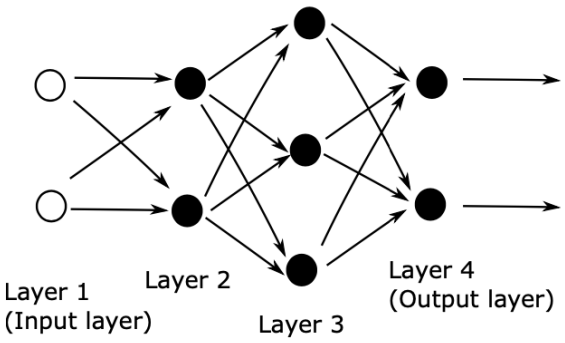


Figura 1: Rete neurale feed-forward.

1. **Input layer**, è il primo layer della rete e non svolge alcuna trasformazione sui dati; il suo unico compito è quello di ricevere il vettore di input e fornirlo in ingresso ai neuroni del primo layer nascosto;
2. **Hidden layers**, sono i layer effettivi della rete come descritti in precedenza;
3. **Output layer**, è l'ultimo layer della rete e produce il risultato finale del modello. Anch'esso è un layer effettivo e dunque performa l'ultima trasformazione affine seguita da un'attivazione.

Definizione 2 (Rete neurale feed-forward)**.** Sia $L \in \mathbb{N}$ il numero di layer e sia n_l il numero di neuroni del layer l, con $n_1 = \dim(\mathbf{x})$. Indichiamo l'input layer con $h^{[1]}(\mathbf{x}) = \mathbf{x}$ e il layer l-esimo, $l = 2, \dots, L$, con la funzione $h^{[l]} \colon \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}$ definita da

$$h^{[l]}(\mathbf{x}) = \sigma^{[l]}(W^{[l]}\mathbf{x} + \mathbf{b}^{[l]}),$$

dove $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ e $\mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$ sono la matrice dei pesi e il vettore dei bias, e la funzione di attivazione $\sigma^{[l]}$ è applicata componente per componente. Si dice *rete neurale feed-forward* la funzione $F \colon \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$ definita da

$$F(\mathbf{x}) = h^{[L]} \circ h^{[L-1]} \circ \cdots \circ h^{[1]}(\mathbf{x}).$$

Possiamo dunque rappresentare l'output della rete come

$$F(\mathbf{x}) = \sigma^{[L]}(W^{[L]}\sigma^{[L-1]}(W^{[L-1]}\sigma^{[L-2]}(\dots)) + \mathbf{b}^{[L]}),$$

oppure ricorsivamente se definiamo con $\mathbf{a}^{[l]}$ l'output del layer l-esimo

$$\begin{aligned} \mathbf{a}^{[1]} &= \mathbf{x} \in \mathbb{R}^{n_1} \\ \mathbf{a}^{[l]} &= \sigma^{[l]}(W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}) \in \mathbb{R}^{n_l}. \end{aligned}$$

Una volta definita la rete come funzione $F \colon \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$, possiamo considerarla nel contesto di un problema di apprendimento supervisionato. Consideriamo lo spazio di probabilità $(\Omega, \mathcal{A}, \mathbb{P})$ e una coppia di variabili aleatorie $(\mathbf{X}, \mathbf{Y}) : \Omega \to \mathcal{X} \times \mathcal{Y}$, dove $\mathcal{X}$ è lo spazio delle features (input) e $\mathcal{Y}$ è lo spazio delle labels (output), la cui legge

$$\rho(B) = \mathbb{P}((\mathbf{X}, \mathbf{Y})^{-1}(B)), \quad B \text{ misurabile},$$

è ignota. Supponiamo ora di disporre di un dataset di N coppie $(\mathbf{x}_j, \mathbf{y}_j)_{j=1}^N$ che sappiamo essere realizzazioni indipendenti di $(\mathbf{X}, \mathbf{Y})$. L'obiettivo dell'approccio supervisionato è quello di costruire una funzione predittiva $F : \mathcal{X} \to \mathcal{Y}$, tale che $F(\mathbf{X}) \approx \mathbf{Y}$. Nel nostro caso, una volta fissati gli iperparametri $L, n_2, \dots, n_L$, tale funzione viene cercata nello spazio parametrico delle reti neurali feed-forward $F \colon \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$ così fatte, di parametri $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}, \mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$, per $l = 2, \dots, L$. Si ottimizzano quindi tali parametri con l'ausilio di una *funzione costo* (o *loss*), ossia una funzione

$$\mathcal{L} : \mathbb{R}^{n_L} \times \mathbb{R}^{n_L} \to \mathbb{R}_+,$$

dove $\mathcal{L}(\mathbf{z}, \mathbf{y})$ è il costo di predire $\mathbf{z}$ quando la label corretta è $\mathbf{y}$. Seguendo il principio della minimizzazione del rischio empirico, si minimizza il valore atteso della loss function rispetto alla misura empirica

$$\hat{\rho}_N(B) = \frac{1}{N}\sum_{j=1}^{N} \delta_{(\mathbf{x}_j, \mathbf{y}_j)}(B), \quad B \text{ misurabile},$$

dove $\delta_{(\mathbf{x}_j, \mathbf{y}_j)}$ è la misura atomica di Dirac centrata in $(\mathbf{x}_j, \mathbf{y}_j)$. Si ha dunque il modello addestrato $F(\mathbf{x}; \hat{\theta})$, con

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \mathbb{E}_{\hat{\rho}_N}\left[\mathcal{L}(F(\mathbf{X}; \theta), \mathbf{Y})\right] = \arg\min_{\theta \in \Theta} \frac{1}{N}\sum_{j=1}^{N} \mathcal{L}(F(\mathbf{x}_j; \theta), \mathbf{y}_j),$$

dove $\theta = (W^{[l]}, \mathbf{b}^{[l]})_{l=2}^{L}$ e $\Theta = \prod_{l=2}^{L}(\mathbb{R}^{n_l \times n_{l-1}} \times \mathbb{R}^{n_l})$. Vale la pena notare che nella letteratura spesso si usa il termine costo per indicare il rischio empirico.

1.1 Ottimizzazione della funzione costo

Da questo momento in poi indicheremo con $\text{Cost}(\theta)$ la media della loss calcolata sul dataset (rischio empirico) e $\mathcal{L}_{\mathbf{x}_j}(\theta) = \mathcal{L}(F(\mathbf{x}_j; \theta), \mathbf{y}_j)$, ossia la loss relativa alla coppia $(\mathbf{x}_j, \mathbf{y}_j)$, e le considereremo come funzioni dei parametri. Una possibile scelta per la loss può essere la seguente

$$\mathcal{L}_{\mathbf{x}_j}(\theta) = \frac{1}{2}\|F(\mathbf{x}_j; \theta) - \mathbf{y}_j\|_2^2, \quad j = 1, \dots, N,$$
$$\text{Cost}(\theta) = \frac{1}{N}\sum_{j=1}^{N} \mathcal{L}_{\mathbf{x}_j}(\theta).$$

È facile intuire che, essendo funzione dei parametri della rete, si tratti di una funzione di molte variabili, il che rende i metodi di ottimizzazione del secondo ordine, come Newton, troppo costosi a livello computazionale. Per ovviare a questo problema introduciamo i metodi del gradiente e del gradiente stocastico. Chiamiamo s il numero di parametri e sia $\theta \in \mathbb{R}^s$. Vogliamo scegliere $\Delta\theta \in \mathbb{R}^s$ tale che $\text{Cost}(\theta + \Delta\theta) < \text{Cost}(\theta)$. Se consideriamo $\Delta\theta$ piccolo e se ignoriamo i termini di ordine $\|\Delta\theta\|_2^2$, la formula di Taylor dice che

$$\text{Cost}(\theta + \Delta\theta) \approx \text{Cost}(\theta) + \nabla\text{Cost}(\theta)^\top \Delta\theta.$$

A questo punto vogliamo scegliere $\Delta\theta$ tale che $\nabla\text{Cost}(\theta)^\top \Delta\theta$ sia il più negativo possibile. Per la disuguaglianza di Cauchy-Schwarz sappiamo che $\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^s$ si ha che $|\mathbf{x}^\top \mathbf{y}| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$, è quindi chiaro che

$$\arg\min_{\substack{\mathbf{y} \in \mathbb{R}^s \\ \|\mathbf{y}\|_2 = 1}} \mathbf{x}^\top \mathbf{y} = -\mathbf{x}.$$

Di conseguenza scegliamo $\Delta\theta$ avente direzione e verso di $-\nabla\text{Cost}(\theta)$, ma vicino a 0 in norma euclidea per garantire precisione all'espansione di Taylor. Scegliamo $\eta > 0$ (piccolo), noto come *learning rate*, e poniamo

$$\Delta\theta = -\eta\nabla\text{Cost}(\theta).$$

Si ha il *metodo del gradiente*:

Algoritmo 1.1 (Metodo del gradiente)**.** Sia $f\colon \mathbb{R}^s \to \mathbb{R}$ derivabile e sia $\theta_0 \in \mathbb{R}^s$. Per $k = 1, 2, \dots$ si itera

$$\theta_k = \theta_{k-1} - \eta_{k-1}\nabla f(\theta_{k-1}).$$

Notiamo che qui il learning rate è variabile nelle iterate: questa è la formulazione più generale, sebbene nella definizione classica il learning rate solitamente si presenti costante. Inoltre, è bene osservare che il metodo del gradiente non garantisce in generale la convergenza al minimo globale: l'algoritmo infatti può

rimanere intrappolato in eventuali minimi locali, punti di sella o regioni quasi piatte. Applicando l'algoritmo al nostro problema e considerando un learning rate costante otteniamo

$$\begin{aligned}\theta_k &= \theta_{k-1} - \eta\nabla\mathrm{Cost}(\theta_{k-1}) \\ &= \theta_{k-1} - \eta\nabla\frac{1}{N}\sum_{j=1}^{N}\mathcal{L}_{\mathbf{x}_j}(\theta_{k-1}) \\ &= \theta_{k-1} - \eta\frac{1}{N}\sum_{j=1}^{N}\nabla\mathcal{L}_{\mathbf{x}_j}(\theta_{k-1}).\end{aligned}$$

Questo implica che ad ogni passo l'algoritmo debba calcolare N gradienti, il che può essere proibitivo dal punto di vista computazionale quando si è in presenza di un gran numero di parametri (come nel caso delle reti neurali) e di un ampio training set; le due condizioni, peraltro, si presentano in coppia, dato che modelli con molti parametri richiedono dataset di grandi dimensioni per ridurre il rischio di overfitting. Per questo motivo si introduce il *metodo del gradiente stocastico*, alternativa meno costosa che, nella sua versione più semplice, consiste nel rimpiazzare $\nabla\mathrm{Cost}(\theta_{k-1})$ con uno solo dei $\nabla\mathcal{L}_{\mathbf{x}_j}(\theta_{k-1})$, scelto casualmente.

Algoritmo 1.2 (Metodo del gradiente stocastico)**.** Sia $\mathrm{Cost}\colon \mathbb{R}^s \to \mathbb{R}$ definita da

$$\mathrm{Cost}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\mathcal{L}_{\mathbf{x}_j}(\theta)$$

e sia $\theta_0 \in \mathbb{R}^s$. Per $k = 1, 2, \ldots$

1. Si sceglie casualmente i da $\{1, \ldots, N\}$ estraendo una realizzazione dalla variabile aleatoria $X \sim \mathrm{Uniform}(\{1, \ldots, N\})$;
2. Si aggiorna
$$\theta_k = \theta_{k-1} - \eta\nabla\mathcal{L}_{\mathbf{x}_i}(\theta_{k-1}).$$

Nella versione appena vista, ad ogni passo si campiona da $\{1, \ldots, N\}$ con reinserimento. Una variante comune consiste nel permutare $\{1, \ldots, N\}$ nel nuovo ordine $\{a_1, \ldots, a_N\}$ ed effettuare gli N passi

$$\theta_{k+i-1} = \theta_{k+i-2} - \eta\nabla\mathcal{L}_{\mathbf{x}_{a_i}}(\theta_{k+i-2}), \quad i = 1, \ldots, N.$$

In tal caso si definisce *epoca* il passaggio dell'algoritmo da tutti gli N punti del training set, dopo di che si ripete per $k = 1, 2, \ldots$. Tale versione può essere generalizzata dal *mini-batch gradient descent*:

1. Si seleziona un intero $m < N$, chiamato *batch size*, tale che $N = Km$, con $K \in \mathbb{N}$;
2. Si permuta $\{1, \ldots, N\}$ in $\{a_1, \ldots, a_N\}$;
3. Per ogni $k = 1, 2, \ldots$ si effettuano i K passi
$$\theta_{k+i-1} = \theta_{k+i-2} - \eta\frac{1}{m}\sum_{j=1}^{m}\nabla\mathcal{L}_{\mathbf{x}_{a_{(i-1)m+j}}}(\theta_{k+i-2}), \quad i = 1, \ldots, K.$$

Nel contesto dell'i-esimo passo, l'insieme di punti $(\mathbf{x}_{a_{(i-1)m+j}})_{j=1}^{m}$ prende il nome di *mini-batch*. In questo caso si dice *epoca* il completamento dei K passi che permettono all'algoritmo di "vedere" il dataset completo.

Esempio 1 (Classificazione)**.** Supponiamo di voler addestrare una rete feed-forward che classifichi una nuova osservazione $\mathbf{x} \in [0,1]^2$ come un cerchio $\bigcirc$ oppure una croce $\times$. Consideriamo quindi un dataset $(\mathbf{x}_j, y_j)_{j=1}^{N}$ dove $\mathbf{x}_j \in [0,1]^2$ per ogni $j = 1, \ldots, N$ e l'etichetta y_j è un cerchio $\bigcirc$ oppure una croce $\times$. Introduciamo la funzione *target output*, $y\colon \mathbb{R}^2 \to \mathbb{R}^2$ definita da

$$y(\mathbf{x}) = \begin{cases} \begin{bmatrix}1\\0\end{bmatrix}, & \text{se } \mathbf{x} \in \mathtt{C}, \\ \\ \begin{bmatrix}0\\1\end{bmatrix}, & \text{se } \mathbf{x} \in \mathtt{X}, \end{cases}$$

dove C e X sono gli insiemi dei punti etichettati rispettivamente come ◯ e ×. A questo punto, dato un nuovo vettore input $\mathbf{x} \in [0,1]^2$, sembrerebbe ragionevole classificarlo in base alla componente più alta del vettore output $F(\mathbf{x};\theta)$ (ad esempio, $F_1(\mathbf{x};\theta) > F_2(\mathbf{x};\theta) \implies \mathbf{x} \in \mathtt{C}$, e viceversa). L'obiettivo sarà quindi ottimizzare i parametri di modo che $F(\mathbf{x};\theta) \approx y(\mathbf{x})$. Sia $F : [0,1]^2 \to \mathbb{R}^2$ la rete neurale da addestrare; allora si minimizza la funzione costo

$$\mathrm{Cost}\big(\theta\big) = \frac{1}{N}\sum_{j=1}^{N}\frac{1}{2}\|y(\mathbf{x}_j) - F(\mathbf{x}_j;\theta)\|_2^2,$$

e si ottengono i parametri $\hat{\theta}$: La rete è addestrata.

1.2 Calcolo dei gradienti

Si introduce il metodo standard per il calcolo dei gradienti $(\nabla\mathcal{L}_{\mathbf{x}_j}(\theta))_{j=1}^N$, ovvero l'algoritmo di *backpropagation*. Per semplificare la notazione abbandoniamo l'indice j e scriviamo

$$\mathcal{L}(\theta) = \mathcal{L}(\theta;\mathbf{a}^{[L]},\mathbf{y}) = \mathcal{L}(F(\mathbf{x};\theta),\mathbf{y}).$$

L'obiettivo è trovare

$$\frac{\partial\mathcal{L}}{\partial w_{ik}^{[l]}}, \quad \frac{\partial\mathcal{L}}{\partial b_i^{[l]}}, \quad \forall l,i,k.$$

Si introducono le seguenti variabili ausiliarie:

1. $\mathbf{z}^{[l]} = W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$, ovvero $\mathbf{a}^{[l]} = \sigma(\mathbf{z}^{[l]})$;
2. $\delta^{[l]} = \begin{bmatrix}\delta_1^{[l]}\\ \vdots \\ \delta_{n_l}^{[l]}\end{bmatrix} \in \mathbb{R}^{n_l}$, dove $\delta_i^{[l]} = \dfrac{\partial\mathcal{L}}{\partial z_i^{[l]}}$.

Definiamo ora la notazione per il prodotto componente per componente:

$$\mathbf{u},\mathbf{v} \in \mathbb{R}^n \implies (\mathbf{u}\circ\mathbf{v})_j = u_j v_j.$$

Lemma 1 (Backpropagation)**.** *Si considerino le variabili definite poc'anzi. Si ha*

1. $\delta^{[L]} = \sigma'(\mathbf{z}^{[L]}) \circ (\mathbf{a}^{[L]} - \mathbf{y})$;
2. $\delta^{[l]} = \sigma'(\mathbf{z}^{[l]}) \circ {W^{[l+1]}}^\top \delta^{[l+1]}$;
3. $\dfrac{\partial\mathcal{L}}{\partial b_i^{[l]}} = \delta_i^{[l]}$;
4. $\dfrac{\partial\mathcal{L}}{\partial w_{ik}^{[l]}} = \delta_i^{[l]} a_k^{[l-1]}$.

Per la dimostrazione si rimanda all'Appendice.

1.3 Esperimenti di classificazione

In questa sezione presentiamo alcuni esperimenti di classificazione implementati in Python. Il primo riguarda un problema di classificazione binaria e multiclasse di punti nel quadrato $[0,1]^2$, affrontato sia con una rete neurale feed-forward sviluppata from-scratch sia utilizzando l'API Keras del pacchetto TensorFlow, una delle principali librerie Python per il deep learning. Sempre con Keras implementiamo poi l'esperimento relativo alla classificazione delle cifre scritte a mano del dataset MNIST. Prima di cominciare installiamo i pacchetti necessari.

```
%pip install numpy
%pip install matplotlib
%pip install tensorflow
%pip install tensorflow_datasets
%pip install scikit-learn
```

In questo caso usiamo il comando %`pip install` $<$`nome_pacchetto`$>$ perché stiamo lavorando in un notebook (file .ipynb): in questo modo il pacchetto viene installato nell'ambiente usato dal notebook ed è subito disponibile per l'importazione.

Classificazione dei punti su $[0,1]^2$ from-scratch. Cominciamo con la creazione di alcuni dataset. La seguente cella di codice definisce una funzione che genera un dataset sintetico di punti sul quadrato; l'input `tipologia` serve a scegliere la tipologia di dataset (in questo caso sono implementati solo `due_parabole` e `due_parabole_random`).

```
import numpy as np
def training_set(tipologia):
    P = 10
    if tipologia == 'due_parabole':

        # Numero di punti
        N = (P + 1) ** 2

        # Inizializzazione dei punti e delle classi
        x1 = np.zeros(N)
        x2 = np.zeros(N)
        y  = np.zeros((3, N))

        # Definizione delle due parabole
        p1 = lambda x: 2 * x * (1 - x)
        p2 = lambda x: 2 * x * (1 - x) + 0.5

        # Distribuzione dei punti sulla griglia e assegnazione delle classi
        k = 0
        for i in range(P + 1):
            for j in range(P + 1):
                x1[k] = i / P
                x2[k] = j / P

                if x2[k] > p2(x1[k]):
                    y[:, k] = [1, 0, 0]
                elif x2[k] < p1(x1[k]):
                    y[:, k] = [0, 0, 1]
                else:
                    y[:, k] = [0, 1, 0]
                k = k + 1

    elif tipologia == 'due_parabole_random':

        # Numero di punti
        N = (P + 1) ** 2

        # Inizializzazione dei punti e delle classi
        x1 = np.zeros(N)
        x2 = np.zeros(N)
        y  = np.zeros((3, N))

        # Definizione delle due parabole
        p1 = lambda x: 2 * x * (1 - x)
```

```
        p2 = lambda x: 2 * x * (1 - x) + 0.5

        # Distribuzione casuale dei punti e assegnazione delle classi
        k = 0
        for i in range(P + 1):
            for j in range(P + 1):
                x1[k] = np.random.rand()
                x2[k] = np.random.rand()

                if x2[k] > p2(x1[k]):
                    y[:, k] = [1, 0, 0]
                elif x2[k] < p1(x1[k]):
                    y[:, k] = [0, 0, 1]
                else:
                    y[:, k] = [0, 1, 0]
                k = k + 1

    else:
        raise ValueError("tipologia non riconosciuta: usa 'due_parabole', 'due_parabole_random', ... ")

    return x1, x2, y
```

Con la seguente cella creiamo un grafico del dataset `due_parabole`, mostrato in Figura 2:

```
#CREAZIONE GRAFICO DEL DATASET DUE PARABOLE

import matplotlib.pyplot as plt

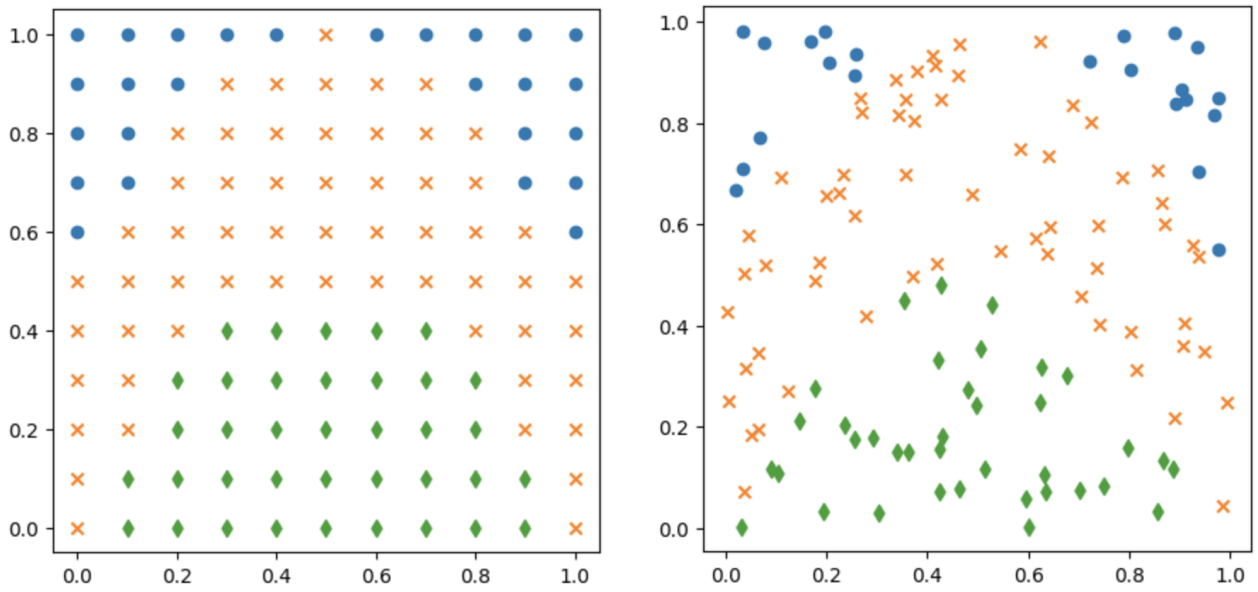
x1, x2, y = training_set('due_parabole')

# Identificazione delle classi
sopra = y[0, :] == 1
centro = y[1, :] == 1
sotto = y[2, :] == 1

# Creazione del grafico
plt.figure(figsize=(5,5))

# Punti
plt.scatter(x1[sopra],   x2[sopra],   marker = 'o')
plt.scatter(x1[centro], x2[centro], marker = 'x')
plt.scatter(x1[sotto],   x2[sotto],   marker = 'd')

plt.show()
```



(a) Training set `due_parabole`.

(b) Training set `due_parabole_random`.

Figura 2

Definiamo ora la funzione di attivazione `my_activate`, di argomenti `x`, `W`, `b`, `activation` (rispettivamente, array di input, array 2D dei pesi, array dei bias e tipologia di attivaizione).

```
def my_activate(x, W, b, activation):
    z = W @ x + b
    if activation == 'linear':
        y = z                       # attivazione
        dy = np.ones(len(z))        # derivata attivazione
    elif activation == 'sigmoid':
        y = 1 / (1 + np.exp(-z))
        dy = y * (1-y)
    elif activation == 'relu':
        y = np.maximum(0, z)
        dy = (z > 0).astype(float)
    else:
        raise ValueError('Attivazione non supportata')
    return [y,dy]
```

Definiamo inoltre il predittore (la rete), ossia la funzione $F\colon \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$ vista in precedenza.

```
# Rete feed-forward
def F(x, Wb, nlv, act_fn):
    a = np.asarray(x).reshape(-1, 1)
    for l in range(1, len(nlv)):
        a, derivate = my_activate(a, Wb['weights'][l], Wb['biases'][l], act_fn)
    return a.reshape(-1)
```

Possiamo quindi scrivere il codice per `nn_train`, funzione che allena una rete neurale feed-forward con mini-batch gradient descent e loss quadratica:

$$\mathrm{Cost}(\theta) = \frac{1}{N}\sum_{j=1}^{N}\frac{1}{2}\|\mathbf{y}_j - F(\mathbf{x}_j;\theta)\|_2^2.$$

Gli argomenti di `nn_train` sono:

- `x1, x2`, array 1D di input di lunghezza `N`;
- `y`, array 2D della forma $(\texttt{nlv}[-1], \texttt{N})$ contenente le labels;
- `act_fn`, funzione di attivazione;
- `nlv`, array 1D o lista di lunghezza `L`, dove `L` è il numero di layer nella rete, le cui entrate sono intere e sono il numero di neuroni presenti in ogni layer. È da notare che $\texttt{nlv}[0]$ e $\texttt{nlv}[-1]$ devono essere uguali rispettivamente a $\texttt{len}(\texttt{np.array}((\texttt{x1}[0], \texttt{x2}[0])))$ e $\texttt{len}(\texttt{y}[:, 0])$;
- `M`, ampiezza del mini-batch;
- `Nepochs`, numero di epoche del mini-batch gradient descent;
- `eta`, learning rate;
- `TOL`, valore soglia per l'early stopping: se durante il training il costo medio scende sotto `TOL`, l'algoritmo interrompe prima di completare tutte le epoche previste.

```
# Definizione e addestramento della rete
def nn_train(x1, x2, y, act_fn, nlv, M, Nepochs, eta, TOL = 1e-4):

    N  = len(x1)            # N vettori di input della forma x = (x1, x2) \in \R^2
    L = len(nlv)            # L è il numero di layers, compreso l'input layer

    Wb = {'weights':[0], 'biases':[0]}

    # Inizializzazione dei parametri
    for l in range(1,L):
        Wb['weights'].append(np.random.normal(0,1,(nlv[l],nlv[l-1])))
        Wb['biases'].append(np.random.normal(0,1,(nlv[l],1)))

    checkpoint = Nepochs // 100       # Ogni quante epoche stampare il costo

    for n in range(Nepochs):

        rkv = np.random.permutation(N)       # Permutazione degli indici 0, ..., N-1

        for b in range(0, N, M):      # 1 completamento del ciclo = 1 epoca
            k = rkv[b:b+M]            # Mini-batch corrente (senza reinserimento)
            if len(k) < M:            # Salta l'ultimo mini-batch se incompleto
                continue

            grad_C = {'weights':[[None for i in range(M)] for j in range(L)],
                      'biases':[[None for i in range(M)] for j in range(L)]}

            for m in range(M):
                x = np.array([x1[k[m]], x2[k[m]]]).reshape(-1, 1)       #data point

                # Forward pass
                a = [None for l in range(L)]
                a[0] = np.asarray(x).reshape(-1, 1)
                D = [None for l in range(L)]
                D[0] = np.identity(len(x))

                for l in range(1,L):
                    act_y, act_dy = my_activate(x = a[l-1], W = Wb['weights'][l],
                                                b = Wb['biases'][l], activation = act_fn)

                    a[l]   = np.asarray(act_y).reshape(-1, 1) # attivazioni come colonna
```

```
                act_dy = np.asarray(act_dy).reshape(-1)    # 1-D prima di fare diag
                D[l]   = np.diag(act_dy)

            # Backward pass
            delta = [None for l in range(L)]
            delta[L-1] = D[L-1] @ (a[L-1] - y[:, k[m]].reshape(-1, 1))

            for l in range(L-2, 0, -1):
                delta[l] = D[l] @ (Wb['weights'][l+1].T @ delta[l+1])

            # Salviamo i gradienti
            for l in range(L-1, 0, -1):
                grad_C['weights'][l][m] = delta[l] @ a[l-1].T
                grad_C['biases'][l][m] = delta[l]

        # Gradient step
        sum_grad_C = {'weights':[None for i in range(L)],
                      'biases':[None for i in range(L)]}
        mean_grad_C = {'weights':[None for i in range(L)],
                      'biases':[None for i in range(L)]}

        for l in range(1,L):
            sum_grad_C['weights'][l] = np.zeros((nlv[l], nlv[l-1]))
            sum_grad_C['biases'][l] = np.zeros((nlv[l], 1))

            for m in range(M):
                sum_grad_C['weights'][l] = (
                    sum_grad_C['weights'][l]
                    + grad_C['weights'][l][m]
                )
                sum_grad_C['biases'][l] = (
                    sum_grad_C['biases'][l]
                    + grad_C['biases'][l][m]
                )

            mean_grad_C['weights'][l] = sum_grad_C['weights'][l]/M
            mean_grad_C['biases'][l] = sum_grad_C['biases'][l]/M

            #Optimization step: aggiornamento dei parametri
        for l in range(1,L):
            Wb['weights'][l] = Wb['weights'][l] - eta * mean_grad_C['weights'][l]
            Wb['biases'][l] = Wb['biases'][l] - eta * mean_grad_C['biases'][l]

    #cost_sum = 0.0

    #for i in range(N):
        #a = F([x1[i], x2[i]], Wb, nlv, act_fn)
        #diff = a - y[:, i].reshape(-1, 1)
        #cost_sum = cost_sum + 0.5 * np.sum(diff**2)
    #cost_avg = cost_sum / N

    #if n % checkpoint == 0:
        #print("Cost = {}, progresso = {}% ".format(cost_avg, ((n+1) / Nepochs) * 100))

    # EARLY STOPPING
    #if cost_avg <= TOL:
            #break

return Wb
```

L'output `Wb` è un dizionario con chiavi `weights` e `biases`, contenente i parametri addestrati della rete. A questo punto possiamo addestrare la rete: scegliamo il training set `due_parabole` e l'architettura `nlv` = `layer`, dove `layer` = `np.array([len(np.array((x1[0],x2[0]))),4,4,len(y[:,0])])`. Decidiamo inoltre di completare `Nepochs` = `10 ** 4` epoche con `eta` = `0.05` e `M` = `5`.

```
# Addestriamo la rete su due_parabole_random
np.random.seed(0)
x1, x2, y = training_set('due_parabole_random')
layers = np.array([len(np.array((x1[0], x2[0]))), 4, 4, len(y[:,0])])

parametri = nn_train(x1, x2, y, act_fn = 'sigmoid',
              nlv = layers,
              M = 5, Nepochs = 10 ** 4, eta = 0.05)
```

Con la seguente funzione otteniamo la previsione del modello sui dati di training:

```
def previsione(x1, x2, y, Wb, nlv, act_fn):
    N = len(x1)

    y_true  = y.copy()                          # (y.shape[0], N)
    y_score = np.zeros((y.shape[0], N))         # (y.shape[0], N)
    y_pred  = np.zeros((y.shape[0], N))         # (y.shape[0], N)
    for i in range(N):
        output_rete = F([x1[i], x2[i]], Wb, nlv, act_fn).reshape(-1)  # (y.shape[0],)
        y_score[:, i] = output_rete
        y_pred[output_rete.argmax(), i] = 1
    return y_score, y_pred, y_true

y_score, y_pred, y_true = previsione(x1, x2, y, parametri, nlv = layers, act_fn = 'sigmoid')
```

Vediamo le regioni ottenute con:

```
def regioni(x1, x2, y, Wb, nlv, act_fn, F, ngrid = 100):

    # Creazione delle liste vuote da riempire con i punti classificati
    xxA, xxB, xxC = [], [], []

    # Creazione griglia di punti e assegnazione alla classe predetta da F
    for u in np.linspace(0, 1, ngrid):
        for v in np.linspace(0, 1, ngrid):
            a = np.array([u, v]).reshape(-1, 1)             # input 2x1
            out = F(a, Wb, nlv, act_fn).reshape(-1, 1)      # output 3x1
            j = np.argmax(out)                              # 0,1,2, classe predetta
            (xxA if j == 0 else xxB if j == 1 else xxC).append([u, v])

    # Creazione grafico delle regioni di classificazione
    plt.figure(figsize = (6,6))
    if len(xxA) > 0: plt.scatter(np.array(xxA)[:, 0], np.array(xxA)[:, 1], s=5, c='r', marker='.')
    if len(xxB) > 0: plt.scatter(np.array(xxB)[:, 0], np.array(xxB)[:, 1], s=5, c='g', marker='.')
    if len(xxC) > 0 : plt.scatter(np.array(xxC)[:, 0], np.array(xxC)[:, 1], s=5, c='b', marker='.')

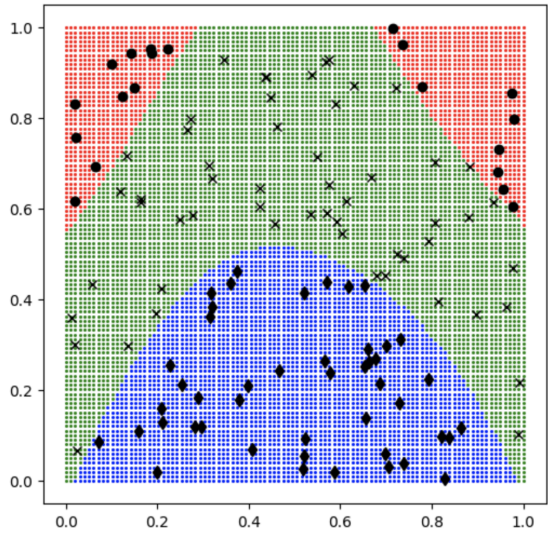
    # Aggiungiamo il dataset al grafico
    for i in range(len(x1)):
        j = np.argmax(y[:, i])  # classe vera
        marker = {0: 'ok', 1: 'xk', 2: 'dk'}[j]
        plt.plot(x1[i], x2[i], marker, linewidth = 2)

    plt.show()
```

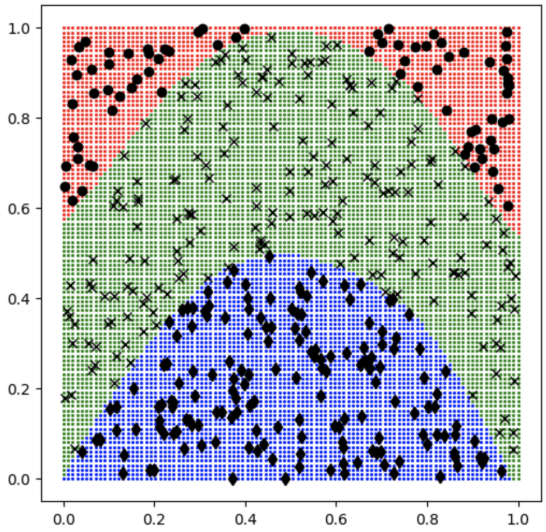
Dopo 10^4 epoche il modello classifica nel modo mostrato in Figura 3(a). Possiamo confrontare le regioni ottenute con quelle che derivano dall'addestramento della rete con `due_parabole`; inoltre, aggiungendo un

layer nascosto e aumentando il numero di neuroni per layer, oltre ad aumentare il numero di punti del traning set da `N = 121` a `N = 441`, si osserva un leggero miglioramento rispetto al caso iniziale, come si può vedere in Figura 3(b) e in Figura 3(d). Per mostrare le regioni chiamiamo la funzione `regioni` dopo aver ottenuto i parametri della rete addestrata.

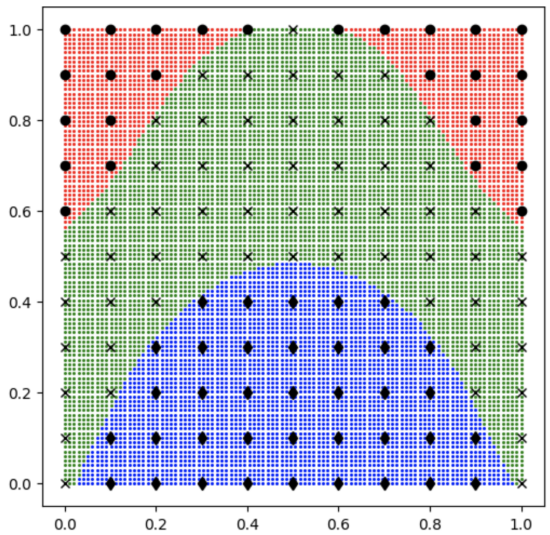
```
# Regioni di classificazione
regioni(x1, x2, y, parametri, nlv = layers, act_fn = 'sigmoid', F = F, ngrid = 100)
```



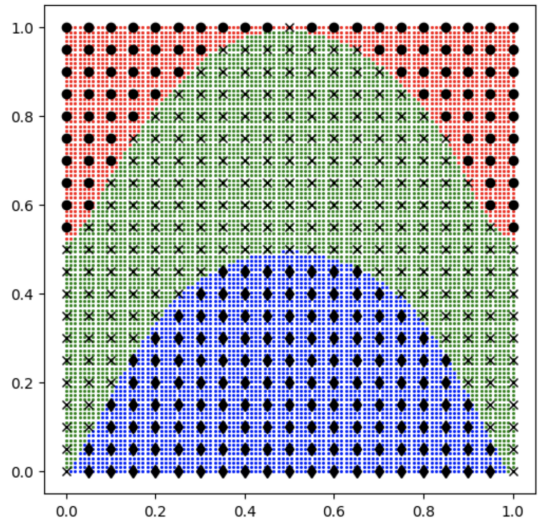
(a) `due_parabole_random`, `N` $= 121$, `nlv` $= [2, 4, 4, 3]$



(b) `due_parabole_random`, `N` $= 421$, `nlv` $= [2, 5, 9, 8, 3]$



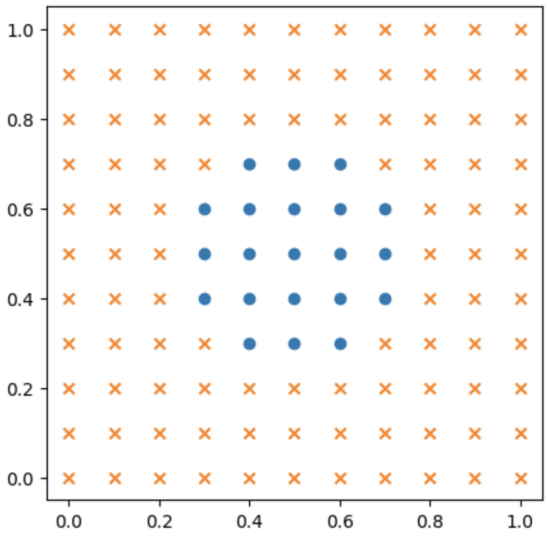
(c) `due_parabole`, `N` $= 121$, `nlv` $= [2, 4, 4, 3]$



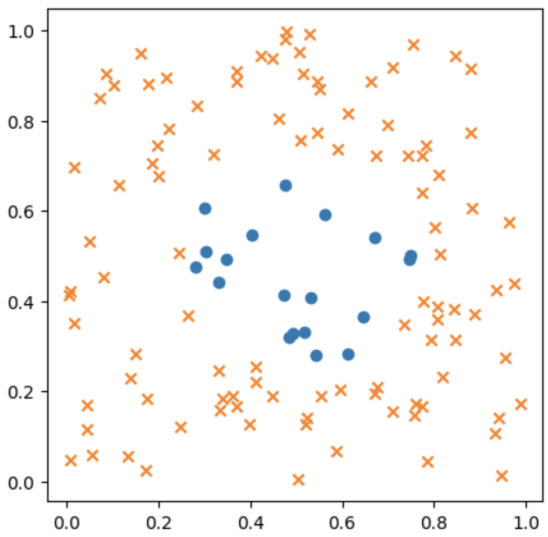
(d) `due_parabole`, `N` $= 421$, `nlv` $= [2, 5, 9, 8, 3]$

Figura 3

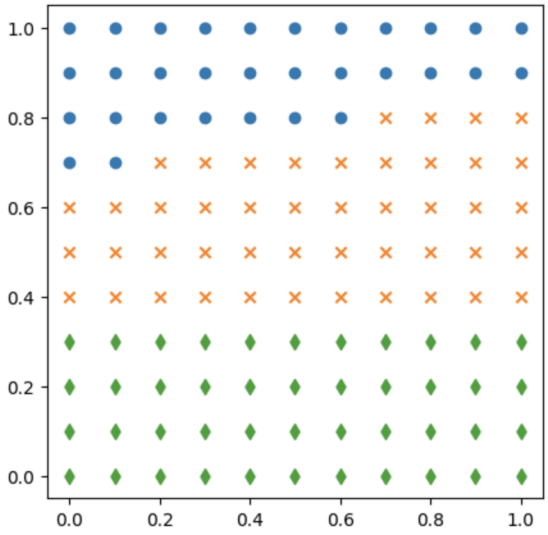
Consideriamo alcuni dataset analoghi a quelli visti in precedenza: `cerchio`, `cerchio_random`, `due_rette`, `due_rette_random`, mostrati in Figura 4. Se `nlv` $=$ `np.array`$(2, 5, 9, 8,$ `len`$($`y`$[:, 0])])$ e `N` $= 441$ si ottiene la classificazione in Figura 5.



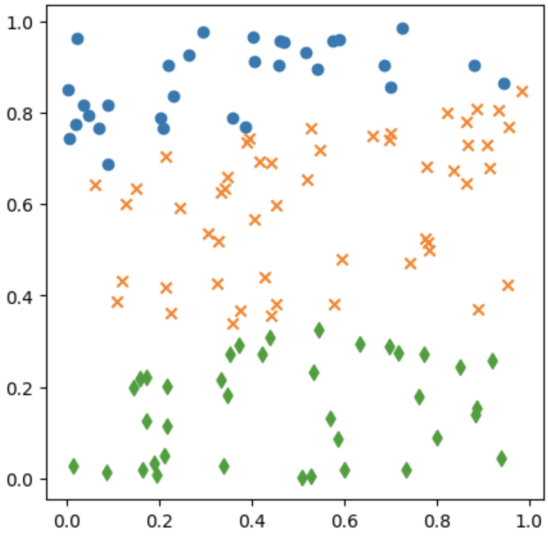
(a) Training set `cerchio`



(b) Training set `cerchio_random`



(c) Training set `due_rette`



(d) Training set `due_rette_random`

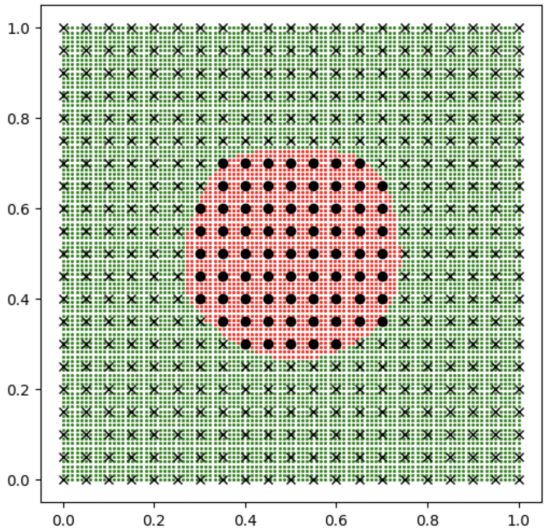
Figura 4

Per monitorare le performance possiamo stampare l'accuratezza della previsione sul training set e la loss finale:

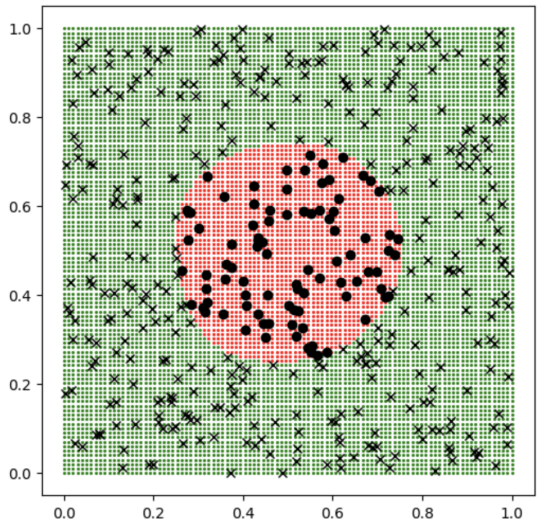
```
# Accuracy sul training set
accuracy = np.mean(y_pred.argmax(axis = 0) == y_true.argmax(axis = 0))
print(accuracy)

costo_finale = (1/len(x1)) * 0.5 * np.sum((([F([x1[i], x2[i]], parametri, layers, 'sigmoid')
                                        for i in range(len(x1))]) - y_true.T) ** 2)
print(costo_finale)
```

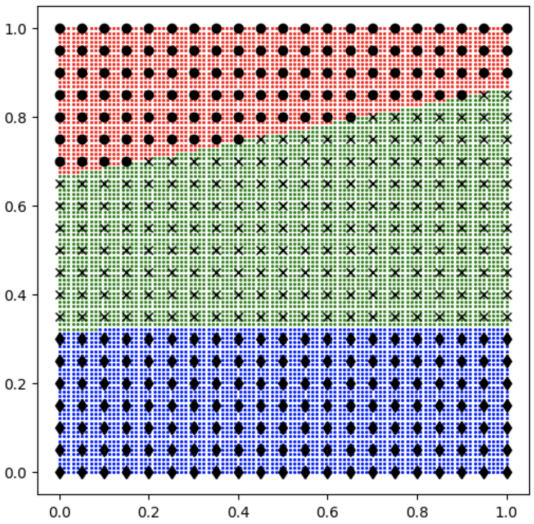
Mostriamo i risultati ottenuti nella Tabella 1. È importante notare che la loss e l'accuracy calcolate sui dati di training non forniscono informazioni sulla capacità di generalizzazione del modello. Per questo motivo, valutiamo il modello anche sulla griglia utilizzata per rappresentare le regioni di classificazione, riportando la relativa loss e accuracy. Definiamo la funzione `grid_label` che assegna ad ogni punto della griglia l'etichetta corretta:



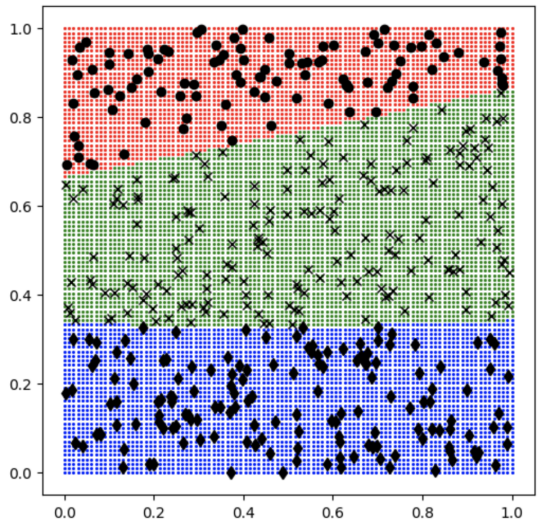
(a) Training set `cerchio`



(b) Training set `cerchio_random`



(c) Training set `due_rette`



(d) Training set `due_rette_random`

Figura 5

```
def grid_label(tipologia, u, v):
    if tipologia == 'due_parabole' or tipologia == 'due_parabole_random':
        p1 = lambda x: 2 * x * (1 - x)
        p2 = lambda x: 2 * x * (1 - x) + 0.5
        if v > p2(u):
            return np.array([1., 0., 0.])
        elif v < p1(u):
            return np.array([0., 0., 1.])
        else:
            return np.array([0., 1., 0.])

    if tipologia == 'due_rette' or tipologia == 'due_rette_random':
        p1 = lambda x: 1/3
        p2 = lambda x: 2/3 + 0.2 * x
        if v > p2(u):
            return np.array([1., 0., 0.])
        elif v < p1(u):
            return np.array([0., 0., 1.])
        else:
            return np.array([0., 1., 0.])

    if tipologia == 'cerchio' or tipologia == 'cerchio_random':
        xc, yc, r = 0.5, 0.5, 0.25
        if (u - xc) ** 2 + (v - yc) ** 2 < r ** 2:
            return np.array([1., 0.])
        else:
            return np.array([0., 1.])

    raise ValueError("Tipologia non supportata")
```

Successivamente definiamo `grid_accuracy` e `grid_loss`:

```
# Accuracy e loss sulla griglia

def grid_accuracy(tipologia, Wb, nlv, act_fn, F, ngrid=100):
    us = np.linspace(0, 1, ngrid)
    vs = np.linspace(0, 1, ngrid)

    corrette = 0
    for u in us:
        for v in vs:
            y_pred = np.argmax(F((u, v), Wb, nlv, act_fn))
            y_true = np.argmax(grid_label(tipologia, u, v))
            corrette = corrette + (y_pred == y_true)
    return corrette / (ngrid * ngrid)

# Loss sulla griglia
def grid_loss(tipologia, Wb, nlv, act_fn, F, ngrid=100):
    us = np.linspace(0, 1, ngrid)
    vs = np.linspace(0, 1, ngrid)

    loss = 0
    for u in us:
        for v in vs:
            differenza = F((u, v), Wb, nlv, act_fn).reshape(-1) - grid_label(tipologia, u, v)
            loss = loss + 0.5 * np.sum(differenza ** 2)
    return loss / (ngrid * ngrid)
```

Si stampano poi a schermo eseguendo i seguenti comandi (ad esempio, per i dati `'due_parabole'`):

```
# Accuracy sulla griglia

print(grid_accuracy('due_parabole', parametri, layers, 'sigmoid', F = F, ngrid = 100))

print(grid_loss('due_parabole', parametri, layers, 'sigmoid', F = F, ngrid = 100))
```

Mostriamo nella Tabella 2 i risultati ottenuti sui sei dataset.

Dataset	N	nlv	Acc	Loss
due_parabole	421	$[2, 5, 9, 8, 3]$	0.9932	0.0051
due_parabole_random	421	$[2, 5, 9, 8, 3]$	0.9977	0.0012
due_parabole	121	$[2, 4, 4, 3]$	0.9917	0.0041
due_parabole_random	121	$[2, 4, 4, 3]$	1.0	0.0
cerchio	421	$[2, 5, 9, 8, 2]$	0.9954	0.0024
cerchio_random	421	$[2, 5, 9, 8, 2]$	0.9977	0.0012
due_rette	421	$[2, 5, 9, 8, 3]$	1.0	0.0
due_rette_random	421	$[2, 5, 9, 8, 3]$	1.0	0.0

Tabella 1: Risultati di accuracy e loss per i diversi dataset e configurazioni della rete sul training set.

Dataset	N	nlv	Acc	Loss
due_parabole	421	$[2, 5, 9, 8, 3]$	0.9833	0.0121
due_parabole_random	421	$[2, 5, 9, 8, 3]$	0.9859	0.0103
due_parabole	121	$[2, 4, 4, 3]$	0.9731	0.0237
due_parabole_random	121	$[2, 4, 4, 3]$	0.9567	0.0359
cerchio	421	$[2, 5, 9, 8, 2]$	0.9829	0.0125
cerchio_random	421	$[2, 5, 9, 8, 2]$	0.9921	0.0060
due_rette	421	$[2, 5, 9, 8, 3]$	0.9983	0.0032
due_rette_random	421	$[2, 5, 9, 8, 3]$	0.9939	0.0049

Tabella 2: Risultati di accuracy e loss per i diversi dataset e configurazioni della rete sulla griglia.

Classificazione dei punti su $[0, 1]^2$ con Keras. In questo paragrafo vediamo come implementare quanto visto nel paragrafo precedente utilizzando l'API di alto livello Keras integrata in TensorFlow. Dopo aver installato TensorFlow importiamo i pacchetti:

```
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
```

Il modo più semplice per creare una rete feedforward è utilizzare la classe `Sequential` che crea una rete neurale vuota con struttura sequenziale. Successivamente aggiungiamo i layer con il metodo `.add()` e chiamiamo il metodo `.compile()` che configura il modello per l'allenamento, specificando ottimizzatore, funzione di perdita e metriche di valutazione. Definiamo una funzione `keras_nn` con gli stessi argomenti visti in precedenza: `nlv`, `act_fn`, `eta`, `loss`, con l'aggiunta di `optimizer_name` per scegliere l'algoritmo di ottimizzazione.

```
# Rete feedforward
def keras_nn(nlv, act_fn, optimizer_name, eta, loss):

    model = keras.models.Sequential()

    # Input layer
    model.add(keras.layers.Input(shape=(nlv[0],)))
```

```
    init = keras.initializers.GlorotUniform()
    #alternativa: init = keras.initializers.RandomNormal(mean = 0.0, stddev = 1.0)

    for neuroni in nlv[1:-1]:
        model.add(keras.layers.Dense(neuroni,
                                     activation = act_fn,
                                     kernel_initializer = init,
                                     bias_initializer = init))     # Hidden layers

    model.add(keras.layers.Dense(nlv[-1],
                                 activation = 'softmax',
                                 kernel_initializer = init,
                                 bias_initializer = init))     # Output layer

    # Definiamo gli optimizers
    if optimizer_name == "SGD":
        optimizer = keras.optimizers.SGD(learning_rate=eta)

    elif optimizer_name == "Adam":
        optimizer = keras.optimizers.Adam(learning_rate=eta)

    else:
        raise ValueError("Aggiungere {} a keras_nn".format(optimizer_name))

    # Configurazione pre-training del modello
    model.compile(optimizer = optimizer,
                  loss = loss,
                  metrics = ['CategoricalAccuracy'])

    return model
```

In questo modo abbiamo creato l'istanza `model` della classe `Sequential` e possiamo dunque procedere all'addestramento della rete chiamando il metodo `.fit()`. Impostiamo il seed del generatore di numeri pseudo-casuali di TensorFlow con `tf.random.set_seed(0)` e formattiamo i dati per renderli utilizzabili come argomenti del metodo `.fit()`. Keras infatti si aspetta i dati di input come array 2D della forma $(\texttt{N}, \texttt{d})$, dove `d` è il numero di features, e i dati di output come array 2D $(\texttt{N}, \texttt{classi})$, dove `classi` è il numero di classi (o, più in generale, la dimensione dell'etichetta). Questo va in contrasto con il modo in cui abbiamo definito la funzione `training_set`, che produce tre oggetti: `x1`, `x2`, array 1D della forma $(\texttt{N},)$ e `y`, array 2D della forma $(3, \texttt{N})$, dove 3 è il numero di classi; dobbiamo dunque unire `x1` e `x2` in una matrice e trasporre `y`. Addestriamo quindi il modello sul dataset `due_parabole`, con $\texttt{Nepochs} = 8000$, mini-batch gradient descent con $\texttt{batch_size} = 32$ e $\texttt{eta} = 0.05$, e la solita loss L^2.

```
# Preparazione del dataset
tf.random.set_seed(0)

x1, x2, y = training_set('due_parabole')

x = np.column_stack([x1, x2])
y = y.T

x_tr, x_val, y_tr, y_val = train_test_split(
    x, y, test_size = 0.2, random_state = 0, shuffle = True, stratify = np.argmax(y, axis=1)
)

# Definizione dell'architettura
layers = np.array([len(np.array((x1[0], x2[0]))), 5, 9, 8, len(y[0,:])])

#Definiamo la loss
def my_loss(y_true, y_pred):
```

```
    return 0.5 * tf.reduce_sum((y_pred - y_true) ** 2, axis = -1)      # tf.reduce_sum() <--> np.sum

# Definizione del modello
model = keras_nn(nlv = layers,
                 act_fn = 'sigmoid',
                 optimizer_name = 'SGD',
                 eta = 0.05,
                 loss = my_loss)

Nepochs = 8000

# Eventuale early stopping
early_stopping = keras.callbacks.EarlyStopping(
    monitor = 'val_CategoricalAccuracy',       # metrica da controllare
    mode = 'max',
    patience = 700,              # quante epoche tollerare senza miglioramento
    restore_best_weights = True   # ripristina i pesi migliori
)
```

Chiamiamo quindi il metodo `.fit()` per addestrare la rete. In questo modo viene restituito un'istanza della classe `keras.callbacks.History` che memorizza, epoca per epoca, i valori di loss e metriche.

```
# Addestramento
training = model.fit(x_tr, y_tr,
                     validation_data = (x_val, y_val),
                     batch_size = 32,
                     epochs = Nepochs,
                     shuffle = True,
                     callbacks = [early_stopping])
```

L'attributo `.history` permette di accedere a un dizionario che contiene le metriche raccolte durante il training. Possiamo plottare `loss`, `val_loss`, `CategoricalAccuracy`, `val_CategoricalAccuracy` in funzione dell'epoca:

```
plt.plot(training.history['loss'], label = 'Training loss')
plt.plot(training.history['val_loss'], label = 'Validation loss')
plt.plot(training.history['CategoricalAccuracy'], label = 'Training accuracy')
plt.plot(training.history['val_CategoricalAccuracy'], label = 'Validation accuracy')

plt.legend()
plt.show()
```

Otteniamo il grafico in Figura 6. I valori finali sono mostrati nella Tabella 3.

loss	0.0517
val_loss	0.0608
CategoricalAccuracy	0.9631
val_CategoricalAccuracy	0.9213

Tabella 3: Loss e accuracy del modello addestrato con `due_parabole`.

Prendendo invece, ad esempio, il dataset `cerchio_random`, osserviamo che il modello con la configurazione appena vista non converge. Mostriamo i valori di loss e accuracy di validation per epoca in Figura 8(a). Come si può vedere, `val_loss` e `val_CategoricalAccuracy` si stabilizzano rispettivamente a 0.1861 e a 0.7528 molto presto. Proviamo a fare i seguenti cambiamenti:

- **Optimizer:** Adam;
- **Loss:** Categorical crossentropy;

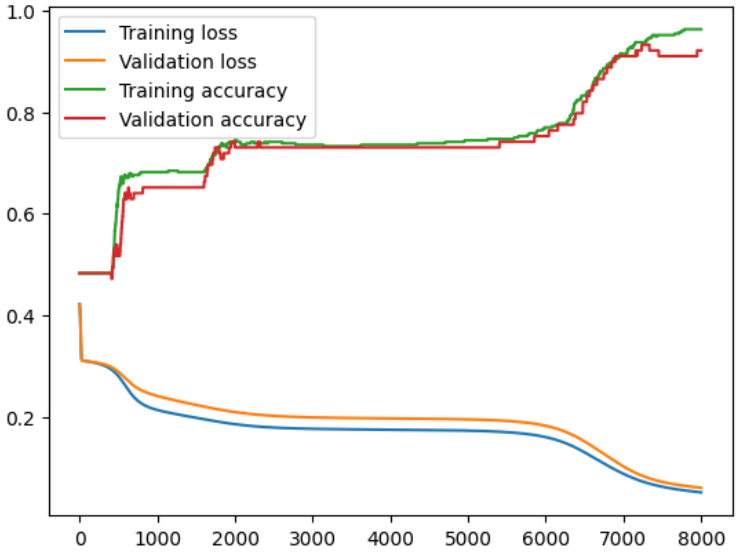


Figura 6: Loss e accuracy per epoca.

- **Attivazione:** ReLu per i layer nascosti, Softmax per il layer di output;
- **Learning rate:** `eta` $= 0.005$.

I risultati sono ottimi: il training si è arrestato all'epoca 1239 per via del callback `early_stopping`, e in Figura 7(b) vediamo che dopo un centinaio di epoche la loss e l'accuracy si attestano già quasi al livello a cui convergono poi più avanti, a meno di alcuni spike dovuti probabilmente alla natura della funzione costo. Questo fenomeno è infatti tipico della categorical crossentropy, definita come

$$\mathcal{L}_{\mathbf{x}_j}(\theta) = -\log\left(F(\mathbf{x}_j;\theta)_{y_j}\right),$$

dove $y_j \in \{1, \ldots, \text{numero di classi}\}$ è la classe reale e $F(\mathbf{x}_j;\theta)_{y_j}$ è la componente y_j-esima del vettore di output della rete. Se la rete assegna una probabilità molto piccola alla classe corretta, cioè $F(\mathbf{x}_j;\theta)_{y_j} \approx 0$, allora il termine $-\log\left(F(\mathbf{x}_j;\theta)_{y_j}\right)$ diventa molto grande e la loss $\mathcal{L}_{\mathbf{x}_j}(\theta)$ subisce un'impennata improvvisa. Nel caso di un validation set ridotto, anche pochi esempi predetti male possono produrre un aumento netto della curva. Dopo il callback `early_stopping` si hanno i risultati mostrati nella Tabella 4.

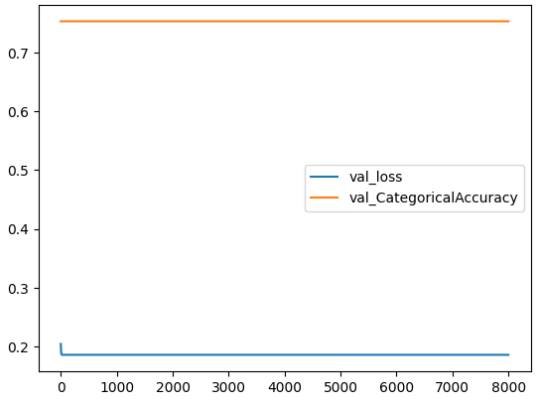
`loss`	0.0026
`val_loss`	0.0307
`CategoricalAccuracy`	1.0
`val_CategoricalAccuracy`	0.9888

Tabella 4: Loss e accuracy su `cerchio_random` con Adam.

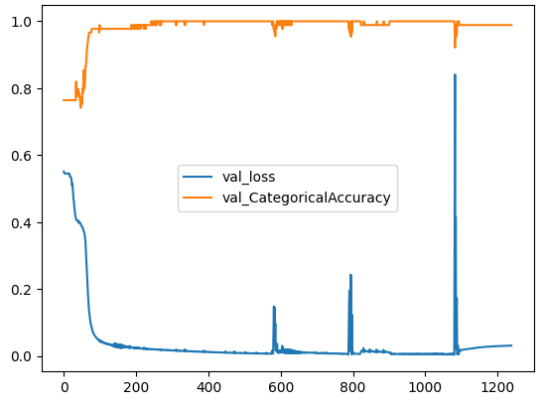
Classificazione delle immagini di MNIST con Keras. Cominciamo importando il dataset direttamente da Keras e normalizzando le immagini dividendo ogni pixel per il valore massimo possibile:

```
# Importiamo i dati
mnist = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

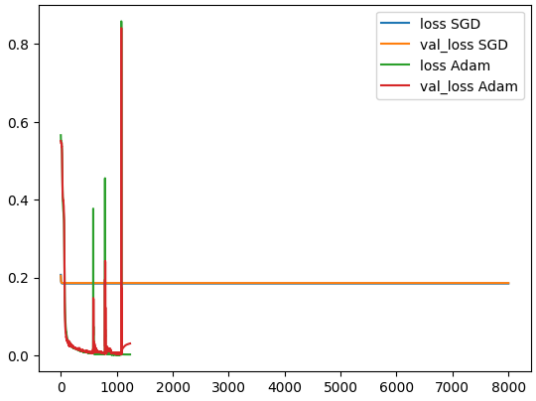
# Normalizzazione
x_train = x_train/255
x_test = x_test/255
```



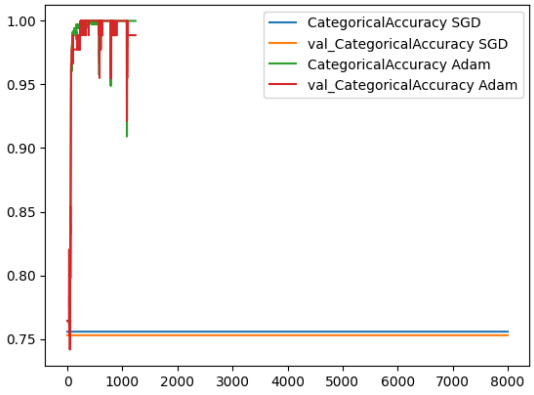
(a) SGD.



(b) Adam.



(c) loss e val_loss comparate.



(d) CatAcc e val_CatAcc comparate.

Figura 7: `cerchio_random`, configurazioni comparate.

Scriviamo l'architettura del modello: ricordiamo che l'inpud deve essere un vettore, quindi dobbiamo "incolonnare" tutti i pixel dell'immagine in input con un layer apposito. Ciò può essere fatto aggiungendo a `model` un oggetto della classe `keras.layers.Flatten`.

```
# Rete feedforward
model = keras.models.Sequential()

model.add(keras.layers.Flatten())  # input layer: appiattisce l'immagine in una colonna
model.add(keras.layers.Dense(128, activation = tf.nn.relu))
model.add(keras.layers.Dense(128, activation = tf.nn.relu))    #2 hidden layers
model.add(keras.layers.Dense(10, activation = tf.nn.softmax))    # output

model.compile(optimizer = 'adam',
              loss = 'sparse_categorical_crossentropy',
              metrics = ['accuracy'])
```

Addestriamo la rete e stampiamo la performance del modello.

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor = 'loss',             # metrica da controllare
    patience = 2,                 # quante epoche tollerare senza miglioramento
    min_delta = 1e-4,
    restore_best_weights = True   # ripristina i pesi migliori
)

Nepochs = 10
training1 = model.fit(x_train, y_train,
                      epochs = Nepochs,
                      batch_size = 32,
                      callbacks = [early_stopping],
                      shuffle = True,
                      validation_data = (x_test, y_test))


val_loss, val_acc = model.evaluate(x_test, y_test)
print(training1.history['loss'][-1], val_loss, training1.history['accuracy'][-1], val_acc)

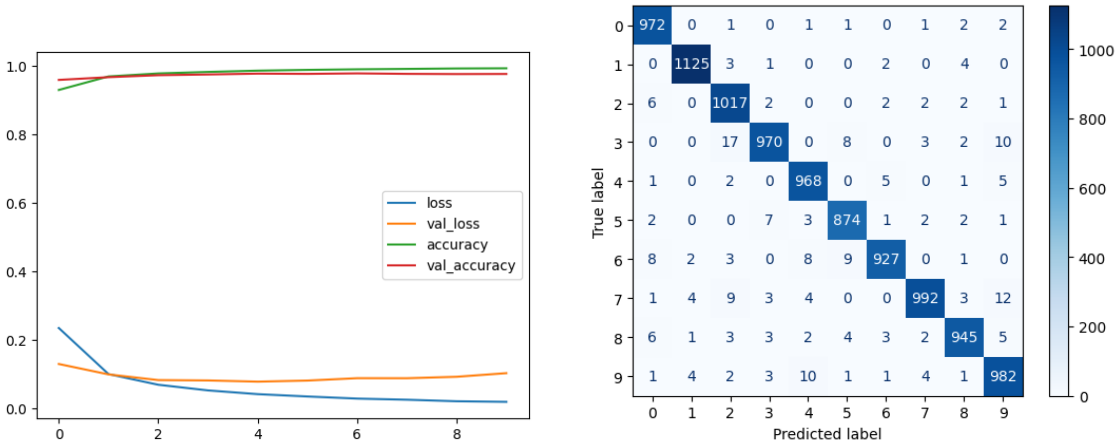
plt.plot(training1.history['loss'], label = 'loss')
plt.plot(training1.history['val_loss'], label = 'val_loss')
plt.plot(training1.history['accuracy'], label = 'accuracy')
plt.plot(training1.history['val_accuracy'], label = 'val_accuracy')

plt.legend()
plt.show()
```

Inoltre con la seguente cella visualizziamo la matrice di confusione, che mostriamo in Figura 8(b) assieme al grafico delle loss e delle accuracy del modello.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, np.argmax(model.predict(x_test), axis = 1))
disp = ConfusionMatrixDisplay(confusion_matrix = cm)
disp.plot(cmap = "Blues")
```



(a) Loss e accuracy.

(b) Matrice di confusione.

Figura 8

2 Reti neurali convoluzionali

Una rete neurale convoluzionale (CNN) è una particolare architettura di rete feed-forward in cui la struttura dei layer è adattata al trattamento di dati con struttura spaziale, come le immagini. La principale novità rispetto al classico *fully connected layer* è rappresentata dal *convolutional layer*: questo calcola convoluzioni bidimensionali tra l'immagine in input e alcuni filtri, i cui parametri vengono appresi durante l'addestramento. In questo modo la rete impara ad estrarre le features utili a compiti come il riconoscimento delle immagini. Per comprendere il funzionamento di questa operazione, cominciamo dalla definizione generale di convoluzione nel contesto continuo.

Definizione 3 (Convoluzione)**.** Siano $f, g\colon \mathbb{R} \to \mathbb{R}$ tali che $f, g \in L^1(\mathbb{R})$. Si definisce *convoluzione* di f e g la funzione

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x-t)\mathrm{d}t.$$

La convoluzione può essere interpretata come una media pesata dei valori di f attorno a x, i cui pesi sono forniti dalla funzione g. Supponiamo che g sia a supporto compatto, $\mathrm{supp}(g) = [-M, M]$: allora si ha che

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x-t)\mathrm{d}t = \int_{-M+x}^{M+x} f(t)g(x-t)\mathrm{d}t.$$

Se $\int_{-M}^{M} g(t)\mathrm{d}t = 1$, allora $(f * g)(x)$ è la media di f in $[x-M, x+M]$ pesata per g. Vediamo un esempio banale.

Esempio 2. Sia $g(t) = \frac{1}{2M}\mathbb{1}_{[-M,M]}(t)$. Allora

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x-t)\mathrm{d}t = \frac{1}{2M}\int_{-\infty}^{\infty} f(t)\mathbb{1}_{[-M,M]}(t)\mathrm{d}t.$$

Dato che $g(x-t) = \begin{cases} \frac{1}{2M} & -M \le x-t \le M \\ 0 & \text{altrimenti} \end{cases} = \begin{cases} \frac{1}{2M} & x-M \le t \le x+M \\ 0 & \text{altrimenti} \end{cases}$ si ha

$$(f * g)(x) = \frac{1}{2M}\int_{-M+x}^{M+x} f(t)\mathrm{d}(t),$$

ossia la media di f su $[x-M, x+M]$.

Nel caso discreto la convoluzione si definisce analogamente.

Definizione 4 (Convoluzione discreta)**.** Siano $f, g\colon \mathbb{Z} \to \mathbb{R}$. La convoluzione discreta di f e g è definita da

$$(f * g)(i) = \sum_{j=-\infty}^{\infty} f(j)\, g(i-j).$$

In questo caso il filtro g è una sequenza di coefficienti tali che $(f * g)(i)$ è una somma pesata dei valori della funzione f nei punti $(i-j)_{j\in\mathbb{Z}}$, con pesi dati da $g(j)$ (in seguito utilizzeremo in maniera intercambiabile le notazioni $g(j)$ e g_j). Se g è a supporto compatto, ad esempio $\operatorname{supp}(g) = \{-1, 0, 1\}$, la sua struttura è completamente determinata dalla sequenza di coefficienti g_{-1}, g_0, g_1. Dopo che avremo definito la convoluzione 2D vedremo come questo formalismo si estende naturalmente al caso bidimensionale.

Riconoscimento delle immagini. Nel contesto della *computer vision*, un'immagine di dimensione $M \times N$ pixel viene rappresentata come una matrice (o meglio, un array) $I \in \mathbb{N}^{M\times N}$, i cui elementi rapresentano le intensità dei singoli pixel. Tale array può essere visto come (e identifica univocamente) una funzione $I\colon \mathbb{Z}^2 \to \mathbb{N}$ a supporto compatto, con $\operatorname{supp}(I) \subseteq \{1, \dots, M\} \times \{1, \dots, N\}$, dove $I(i, j)$ rappresenta l'intensità del pixel in posizione (i, j). Allo stesso modo, un filtro bidimensionale è una matrice di coefficienti $g \in \mathbb{R}^{(2C+1)\times(2C+1)}$ (solitamente si utilizzano filtri di piccole dimensioni, ad esempio 3×3), ossia una funzione $g\colon \mathbb{Z}^2 \to \mathbb{R}$ a supporto compatto, con $\operatorname{supp}(g) \subseteq \{-C, \dots, C\} \times \{-C, \dots, C\}$. Osserviamo che, per come lo abbiamo definito, g è quadrato e ha lato dispari; queste due condizioni non sono in teoria obbligatorie, sebbene, per ragioni che saranno più chiare in seguito, siano di fatto lo standard nei modelli di riconoscimento delle immagini.

Convenzione degli indici. Per l'immagine adotteremo un'indicizzazione a partire da 1: $i \in \{1, \dots, M\}$ denota la riga (dall'alto verso il basso) e $j \in \{1, \dots, N\}$ la colonna (da sinistra verso destra). Per il filtro g usiamo indici centrati $(i, j) \in \{-C, \dots, C\}^2$, dove i è lo spostamento verticale rispetto alla riga centrale (positivo verso il basso) e j quello orizzontale dalla colonna centrale (positivo verso destra).

Definizione 5 (Convoluzione discreta 2D)**.** Siano $f, g\colon \mathbb{Z}^2 \to \mathbb{R}$. La convoluzione discreta bidimensionale di f e g è

$$(f * g)(i, j) = \sum_{l=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} f(l, k) g(i-l, j-k).$$

Data un'immagine I, e supponendo $2C + 1 \leq \min(M, N)$, l'applicazione del filtro g corrisponde quindi al calcolo della convoluzione discreta bidimensionale, a cui viene aggiunto un bias costante:

$$\begin{aligned}(I * g)_{i,j} + b &= \sum_{l=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} I_{l,k} g_{i-l,j-k} + b \\ &= \sum_{l=i-C}^{i+C} \sum_{k=j-C}^{j+C} I_{l,k} g_{i-l,j-k} + b,\end{aligned}$$

così che $(I * g)_{i,j}$ risulta essere una combinazione affine dei valori dei pixel vicini a (i, j) con coefficienti determinati dal filtro g. In questo modo si crea una nuova immagine in cui sono risaltate determinate features. Vediamo un esempio

Esempio 3. Supponiamo di avere un'immagine $I \in \mathbb{R}^{28\times 28}$ e un filtro g rappresentabile come matrice $g \in \mathbb{R}^{3\times 3}$ nel modo seguente:

$$g = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} = \begin{bmatrix} g_{-1,-1} & g_{-1,0} & g_{-1,1} \\ g_{0,-1} & g_{0,0} & g_{0,1} \\ g_{1,-1} & g_{1,0} & g_{1,1} \end{bmatrix}.$$

La nuova immagine è definita pixel per pixel dalla convoluzione di I e g più un bias che, in questo caso, per semplicità, consideriamo nullo:

$$\begin{aligned}(I * g)_{i,j} + b &= \sum_{l=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} I_{l,k} g_{i-l,j-k} \\ &= \sum_{l=i-1}^{i+1} \sum_{k=j-1}^{j+1} I_{l,k} g_{i-l,j-k} \\ &= I_{i-1,j-1} g_{1,1} + I_{i-1,j} g_{1,0} + I_{i-1,j+1} g_{1,-1} \\ &+ I_{i,j-1} g_{0,1} + I_{i,j} g_{0,0} + I_{i,j+1} g_{0,-1} \\ &+ I_{i+1,j-1} g_{-1,1} + I_{i+1,j} g_{-1,0} + I_{i+1,j+1} g_{-1,-1} \\ &= 0 - I_{i-1,j} + 0 \\ &- I_{i,j-1} + 4I_{i,j} - I_{i,j+1} \\ &+ 0 - I_{i+1,j} + 0, \quad \forall (i,j) \in \{1,\dots,M\} \times \{1,\dots,N\}.\end{aligned}$$

Osserviamo che, nell'espressione precedente, i coefficienti del filtro g che moltiplicano i valori di I non compaiono con la stessa disposizione della matrice originaria, ma con le coordinate invertite. È infatti comune in computer vision calcolare la convoluzione tra I e g', dove g' è il filtro g a coordinate invertite (ossia ruotato di 180°), definito da $g'(i,j) = g(-i,-j)$. In questo modo si ha

$$(I * g')_{i,j} = \sum_{l=i-1}^{i+1} \sum_{k=j-1}^{j+1} I_{l,k} g'_{i-l,j-k} = \sum_{l=i-1}^{i+1} \sum_{k=j-1}^{j+1} I_{l,k} g_{l-i,k-j},$$

che nell'Esempio 3 si traduce nei seguenti passaggi.

Esempio 3 (Continua)**.**

$$\begin{aligned}(I * g')_{i,j} &= \sum_{l=i-1}^{i+1} \sum_{k=j-1}^{j+1} I_{l,k} g'_{i-l,j-k} \\ &= \sum_{l=i-1}^{i+1} \sum_{k=j-1}^{j+1} I_{l,k} g_{l-i,k-j} \\ &= I_{i-1,j-1} g_{-1,-1} + I_{i-1,j} g_{-1,0} + I_{i-1,j+1} g_{-1,1} \\ &+ I_{i,j-1} g_{0,-1} + I_{i,j} g_{0,0} + I_{i,j+1} g_{0,1} \\ &+ I_{i+1,j-1} g_{1,-1} + I_{i+1,j} g_{1,0} + I_{i+1,j+1} g_{1,1} \\ &= 0 - I_{i-1,j} + 0 \\ &- I_{i,j-1} + 4I_{i,j} - I_{i,j+1} \\ &+ 0 - I_{i+1,j} + 0, \quad \forall (i,j) \in \{1,\dots,M\} \times \{1,\dots,N\}.\end{aligned}$$

Osserviamo che in questo caso $(I * g)_{i,j} = (I * g')_{i,j}$, infatti $g_{i,j} = g_{-i,-j},\ \forall (i,j) \in \{-C,\dots,C\}^2$.

Padding. Nell'Esempio 3 si è asserito che l'immagine risultante dalla convoluzione sia definita pixel per pixel per ogni $(i,j) \in \{1,\dots,M\} \times \{1,\dots,N\}$. Tuttavia, quando il filtro è centrato su un pixel troppo vicino al bordo, la finestra di convoluzione eccede il supporto dell'immagine e alcuni termini della somma non sono definiti (o meglio, se interpretiamo I come una funzione, tali termini sono esattamente 0; se invece la interpretiamo come matrice tali termini sono indefiniti). In particolare, se consideriamo un filtro quadrato di $(2C+1) \times (2C+1)$, osserviamo che le prime e le ultime C righe e le prime e le ultime C colonne di I sono oggetto di questo fenomeno. Per ovviare al problema si introduce il *padding*, ossia un'estensione artificiale dell'immagine oltre i suoi confini. La scelta più comune è lo *zero-padding*, in cui i pixel aggiunti sono posti pari a zero, che corrisponde a considerare I una funzione pari a 0 fuori dal'insieme $\{1,\dots,M\} \times \{1,\dots,N\}$, tuttavia in letteratura si trovano anche varianti come il *replication padding* (replica dei valori sul bordo) o

il *reflection padding* (riflessione speculare rispetto al bordo). L'introduzione del padding garantisce quindi che la convoluzione sia ben definita per tutti i pixel e consente di mantenere la stessa dimensione dell'output rispetto all'input.

Convolutional layer. A questo punto possiamo definire con precisione il funzionamento del convolutional layer. Siano $I \in \mathbb{N}^{M\times N}$ l'immagine, $g^{(1)},\dots,g^{(n_1)} \in \mathbb{R}^{(2C+1)\times(2C+1)}$ i filtri e $b^{(1)},\dots,b^{(n_1)}$ i bias; chiamiamo $a_1,\dots,a_{n_1}$ i *neuroni convoluzionali*, definiti da

$$a_n(i,j) = \sigma((I * g^{(n)})(i,j) + b^{(n)}), \quad n = 1,\dots,n_1,$$

dove σ è la funzione di attivazione. In analogia con il layer classico, chiamiamo *layer convoluzionale* l'insieme di n_1 neuroni di convoluzione che lavorano in parallelo prendendo in input la stessa immagine. La funzione di output di un neurone convoluzionale è anche detta *feature map*.

Pooling layer. Prima di vedere un esempio di CNN resta da definire un'ultima operazione fondamentale, ossia il *pooling*. Il pooling è un'operazione che agisce sull'output di ciascun neurone, con lo scopo di ridurne la dimensione, rendere la rappresentazione dei dati iniziali più robusta a piccole distorsioni ed eliminare informazioni ridondanti. Fissata una finestra di dimensioni $p_h \times p_w$ e uno *stride* s, il pooling consiste nello scorrere tale finestra sulla feature map e, ad ogni passo, sostituire il blocco di pixel coperto dalla finestra con un unico valore, calcolato tramite una funzione di aggregazione predefinita (ad esempio il massimo o la media). Lo stride determina di quanti pixel la finestra viene traslata a ogni passo. Nella pratica, il pooling viene solitamente effettuato con finestre quadrate e, per evitare sovrapposizioni, si sceglie spesso $s = p_h = p_w$.

Esempio 4. Chiamiamo $P_{i,j}$ l'insieme di indici coperti dalla finestra di pooling il cui angolo superiore sinistro ha coordinate (i,j), e sia $a(i,j)$ una feature map. Le seguenti sono alcune delle più comuni tecniche di pooling.

- **Max Pooling (MaxPool).** Il Max Pooling mappa ogni finestra al valore del pixel con la massima intensità:
$$\text{MaxPool}(i,j) = \max_{(u,v)\in P_{i,j}} a(u,v)$$
- **Average Pooling (AvgPool).** L'Average Pooling calcola la media aritmetica dei pixel interni alla finestra:
$$\text{AvgPool}(i,j) = \frac{1}{|P_{i,j}|} \sum_{(u,v)\in P_{i,j}} a(u,v)$$
- **Mixed Pooling (MixPool).** Il Mixed Pooling combina Max Pooling e Average Pooling. Fissato $\alpha \in [0,1]$ si ha:
$$\text{MixPool}(i,j) = \alpha\text{MaxPool}(i,j) + (1-\alpha)\text{AvgPool}(i,j).$$
- **Stochastic Pooling.** Lo Stochastic Pooling consiste nel campionare casualmente un pixel interno alla finestra $P_{i,j}$, dove la probabilità di estrarre il pixel (u,v) è $\dfrac{a(u,v)}{\sum_{(l,k)\in P_{i,j}} a(l,k)}$.

Rete neurale convoluzionale. Una volta definito il pooling possiamo vedere un esempio del blocco "tipico" di una CNN. Data l'immagine I in input, innanzitutto si ha un *layer convoluzionale* che ha come output n_1 feature maps. A questo punto segue il *pooling layer*, dove le feature maps vengono rimpicciolite. Successivamente, l'operazione più comune è quella di "appiattire" ciascuna feature map ottenuta dal pooling in un array colonna di dimensione $N \times 1$, dove N rappresenta il numero di pixel rimasti dopo il pooling. Gli array così ottenuti vengono concatenati in un unico vettore, che costituisce l'input per un fully connected layer. L'output di quest'ultimo può quindi essere passato a un ulteriore blocco feedforward, oppure rappresentare direttamente la classificazione finale della rete.

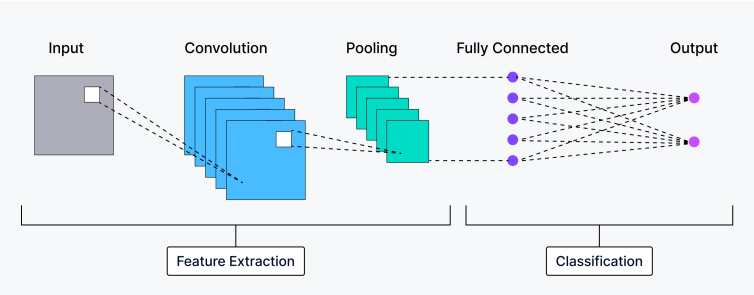


Figura 9: Rete neurale convoluzionale.

2.1 Classificazione di immagini con CNNs

Dopo aver presentato i principi teorici delle reti convoluzionali, passiamo ora alla fase sperimentale, in cui applichiamo le CNN a compiti di classificazione di immagini. L'obiettivo è valutare l'efficacia di queste architetture su dataset standard, confrontando le prestazioni ottenute con quelle delle reti feedforward viste in precedenza.

Classificazione delle immagini di MNIST con le CNN di Keras. Come abbiamo visto in precedenza, MNIST è compreso tra i dataset di Keras. Possiamo quindi importarlo con `keras.datasets.mnist.load_data()`. Procediamo poi a costruire una rete neurale convoluzionale come nell'esempio in Figura 11. Scegliamo l'attivazione `relu`, la loss `sparse_categorical_crossentropy` (ossia l'entropia incrociata nel caso in cui le labels siano interi; nel caso in cui le etichette siano vettori one-hot si usa `categorical_crossentropy`), l'optimizer `adam` e `softmax` come attivazione dell'output layer.

```
# KERAS CONVNET - MNIST

import numpy as np
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

tf.random.set_seed(0)

# Importiamo i dati
mnist = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalizzazione
x_train = x_train/255
x_test = x_test/255

Nlabels = np.max(y_train) + 1        # In questo caso le classi sono {0, ..., Nlabels - 1}

Nepochs = 6

model = keras.models.Sequential()

# Convolutional layer
model.add(keras.layers.Input(shape = (28, 28, 1)))
model.add(keras.layers.Conv2D(64, (3,3)))
model.add(keras.layers.Activation("relu"))

# Pooling layer
model.add(keras.layers.MaxPooling2D(pool_size = (2,2)))
```

```
# Fully connected layer
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(Nlabels))
model.add(keras.layers.Activation("softmax"))

model.compile(loss = "sparse_categorical_crossentropy",
              optimizer = "adam",
              metrics = ["accuracy"])

training_convnet_mnist = model.fit(x_train, y_train,
                                   batch_size = 32,
                                   epochs = Nepochs,
                                   validation_data = (x_test, y_test),
                                   shuffle = True)
```

Con sole 6 epoche le performance sono ottime: `accuracy` : 0.9930, `loss` : 0.0258, `val_accuracy` : 0.9836, `val_loss` : 0.0572. In Figura 10 vediamo che la convergenza è pressochè immediata.

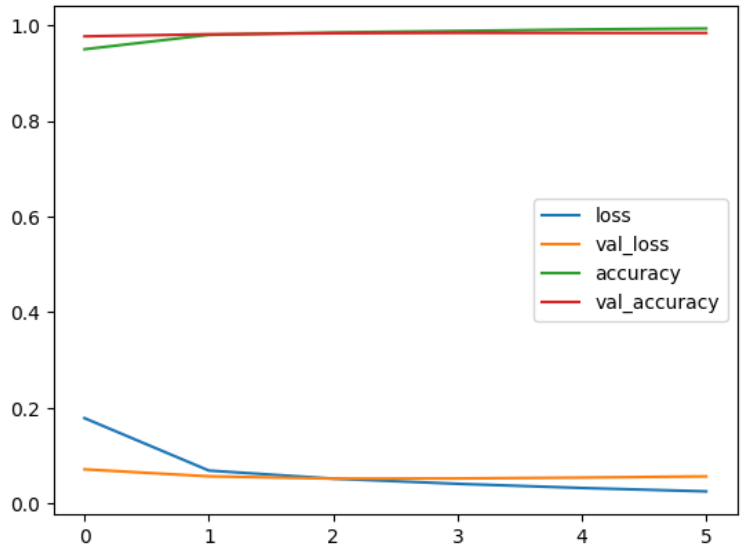


Figura 10: Convergenza di loss e accuracy.

Classificazione delle immagini di CIFAR-10 con le CNN di Keras. Il dataset CIFAR-10 consiste in 60000 immagini 32×32, etichettate in 10 classi, con 6000 immagini per classe. Allo stesso modo di MNIST, CIFAR-10 si può trovare in `keras.datasets`; questa volta però ci troviamo in presenza di immagini a colori. Questo significa che ogni immagine è un'array NumPy di dimensione (altezza , larghezza, 3), dove ogni pixel è un vettore di 3 canali che indicano le intensità dei tre colori rosso, verde e blu. Le intensità sono solitamente interi da 0 (pixel spento) a 255. Per questo motivo è necessario fare una precisazione sul funzionamento della convoluzione con immagini a tre canali. In un'immagine RGB, il layer `Conv2D` usa filtri di forma $k_h \times k_w \times 3$. Per ogni posizione, si moltiplica la finestra $k_h \times k_w$ di ogni canale (R,G,B) con i pesi della corrispondente dimensione del filtro si e sommano i tre contributi (più il bias), producendo una feature map per filtro.

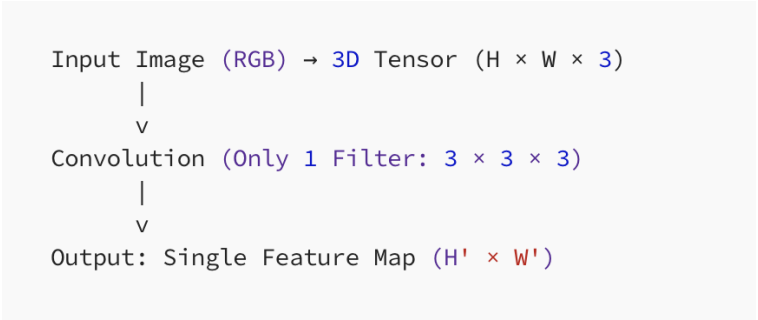


Figura 11: Struttura della convoluzione 2D su immagini RGB, con filtro $(3 \times 3 \times 3)$.

Implementiamo lo stesso modello che abbiamo utilizzato per classificare le immagini di MNIST, adattato per la dimensione dei nuovi dati.

```
#KERAS CONVNET - CIFAR-10

# Importiamo i dati
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# Normalizzazione
x_train = x_train/255
x_test = x_test/255

Nlabels = np.max(y_train) + 1        # In questo caso le classi sono {0, ..., Nlabels - 1}

Nepochs = 6

model = keras.models.Sequential()

# Convolutional layer
model.add(keras.layers.Input(shape = (32, 32, 3)))
model.add(keras.layers.Conv2D(64, (3,3)))
model.add(keras.layers.Activation("relu"))

# Pooling layer
model.add(keras.layers.MaxPooling2D(pool_size = (2,2)))

# Fully connected layer
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(Nlabels))
model.add(keras.layers.Activation("softmax"))

model.compile(loss = "sparse_categorical_crossentropy",
              optimizer = "adam",
              metrics = ["accuracy"])

training_convnet_cifar10_1 = model.fit(x_train, y_train,
                                       batch_size = 32,
                                       epochs = Nepochs,
                                       validation_data = (x_test, y_test),
                                       shuffle = True)
```

Come ci si potrebbe aspettare, data la complessità maggiore dei dati, i risultati sono deludenti in confronto a quelli ottenuti con MNIST: `accuracy` : 0.7115, `loss` : 0.8461, `val_accuracy` : 0.6182, `val_loss` : 1.1295. Raddoppiando il numero di epoche la situazione non migliora granchè, infatti loss e accuracy di validation rimangono pressochè costanti dopo la sesta epoca: `accuracy` : 0.7932, `loss` : 0.6135, `val_accuracy` : 0.6104,

`val_loss` : 1.2736. Proviamo ad aggiungere i seguenti layer tra il primo pooling layer e il fully connected layer:

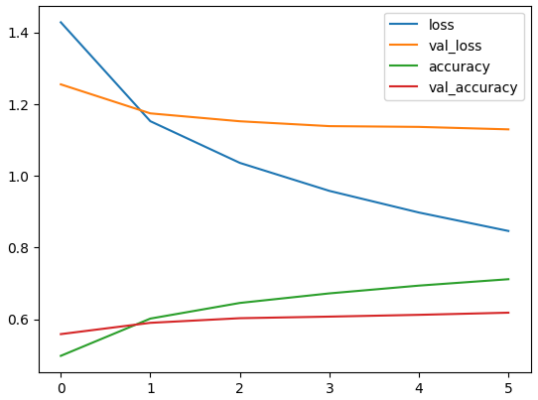
```
# Convolutional layer
model.add(keras.layers.Conv2D(256, (3,3)))
model.add(keras.layers.Activation("relu"))

# Pooling layer
model.add(keras.layers.MaxPooling2D(pool_size = (2,2)))

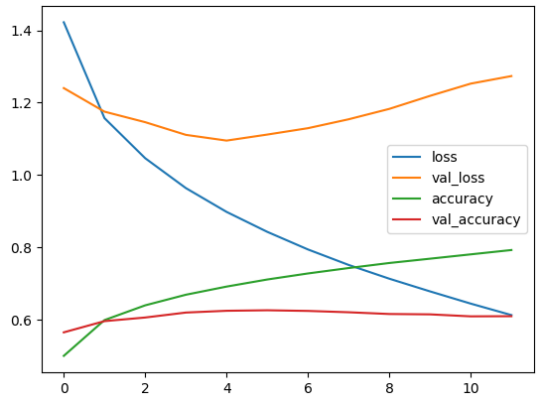
# Convolutional layer
model.add(keras.layers.Conv2D(90, (3,3)))
model.add(keras.layers.Activation("relu"))

# Pooling layer
model.add(keras.layers.MaxPooling2D(pool_size = (2,2)))
```

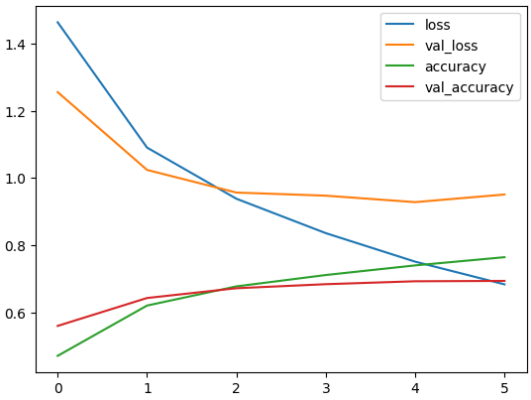
Ancora, nonostante si osservi un miglioramento rispetto alla precedente architettura, rispetto a MNIST si osservano loss e accuracy molto peggiori: `accuracy` : 0.7647, `loss` : 0.6839, `val_accuracy` : 0.6942, `val_loss` : 0.9513. Mostriamo in Figura 12 gli andamenti di loss e accuracy in funzione dell'epoca.



(a) Prima configurazione.



(b) Seconda configurazione.



(c) Terza configurazione.

Figura 12: Convergenza degli indicatori nelle tre configurazioni, dataset CIFAR-10.

Classificazione delle immagini di SVHN con le CNN di Keras. A differenza dei dataset precedenti, SVHN non è parte di `keras.datasets`, bensì del pacchetto `tensorflow_datasets` che dobbiamo quindi

importare. Utilizziamo poi la funzione `tensorflow_datasets.load()` per caricare il dataset. L'argomento `as_supervised` permette di caricare anche le labels.

```
import tensorflow_datasets as tfds

ds_train, ds_test = tfds.load(
    "svhn_cropped",
    split = ["train+extra", "test"],
    as_supervised = True,
)
```

Nel nostro caso `ds_train` e `ds_test` non sono array NumPy, ma istanze di `tf.data.Dataset`. Quando si passa un oggetto `tf.data.Dataset` a `model.fit`, i parametri `batch_size` e `shuffle` di Keras non vengono applicati: l'ampiezza del mini batch e lo shuffling devono essere definiti nella pipeline stessa, tramite i metodi `.shuffle()` e `.batch()` del dataset. Inoltre `.shuffle(buffer_size)` tiene in memoria solo i primi `buffer_size` esempi, estrae ogni volta a caso un elemento da tale sottoinsieme (buffer) come output e lo rimpiazza con il prossimo esempio del dataset rimasto fuori dal buffer. Se `buffer_size` è grande quanto l'intero dataset si ottiene uno shuffle globale; con valori più piccoli si ottiene uno shuffle "locale" (con il vantaggio di un minore uso di memoria RAM). Il rimescolamento si ripete a ogni epoca (`reshuffle_each_iteration = True` di default). Proviamo dunque anche qui ad addestrare il modello formato dal blocco convoluzionale standard.

```
# KERAS CONVNET - SVHN

tf.random.set_seed(0)

train_data = ds_train.shuffle(buffer_size = 10000, reshuffle_each_iteration = True).batch(32)
test_data  = ds_test.batch(32)

Nlabels = 10

Nepochs = 6

model = keras.models.Sequential()

# Convolutional layer
model.add(keras.layers.Input(shape = (32, 32, 3)))
model.add(keras.layers.Rescaling(1./255))                          # normalizzazione
model.add(keras.layers.Conv2D(64, (3,3), padding = 'same'))        # zero-padding
model.add(keras.layers.Activation("relu"))

# Pooling layer
model.add(keras.layers.MaxPooling2D(pool_size = (2,2)))

# Fully connected layer
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(Nlabels))
model.add(keras.layers.Activation("softmax"))

early_stopping = keras.callbacks.EarlyStopping(
    monitor = 'loss',             # metrica da controllare
    patience = 1,                 # quante epoche tollerare senza miglioramento
    min_delta = 1e-4,
    restore_best_weights = True   # ripristina i pesi migliori
)

model.compile(loss = "sparse_categorical_crossentropy",
              optimizer = "adam",
              metrics = ["accuracy"])
```

```
training_convnet_svhn1 = model.fit(train_data,
                                   epochs = Nepochs,
                                   validation_data = test_data,
                                   callbacks = [early_stopping])
```

Le prestazioni sono superiori allo stesso modello addestrato su CIFAR-10: `accuracy` : 0.9055, `loss` : 0.3561, `val_accuracy` : 0.8316, `val_loss` : 0.6529. Aggiungendo un layer di convoluzione + ReLU otteniamo i seguenti risultati: `accuracy` : 0.9192, `loss` : 0.2882, `val_accuracy` : 0.8639, `val_loss` : 0.4942.

3 Conclusioni

In conclusione, in questa relazione si è introdotto il deep learning combinando un approccio teorico con esempi di implementazione in Python. Nella prima parte si sono descritte le reti neurali feed-forward, la minimizzazione della funzione di costo e l'algoritmo di backpropagation per il calcolo dei gradienti. Si è poi visto come i modelli sviluppati from-scratch e con Keras abbiano classificato con alta accuratezza i dataset sintetici su $[0, 1]^2$, ad eccezione del caso `cerchio_random` per il quale è stato necessario adottare l'optimizer Adam e sostituire la loss con l'entropia incrociata, ottenendo così un addestramento stabile e preciso. Nella seconda parte si sono presentate le reti neurali convoluzionali e il loro funzionamento nel contesto del riconoscimento di immagini: i modelli hanno ottenuto performance soddisfacenti su MNIST e SVHN, mentre su CIFAR-10 i risultati non sono stati sufficienti. Utilizzando il lavoro svolto come punto di partenza, la continuazione naturale è estendere l'analisi sperimentale esplorando architetture più profonde e adottando in modo sistematico tecniche più avanzate come regolarizzazione e data augmentation, con l'obiettivo di migliorare le prestazioni sui dataset più complessi.

Appendice

Dimostriamo il lemma di Backpropagation.

Dimostrazione. Ricordiamo che per ogni l:

$$\mathbf{a}^{[l]} = \sigma\big(\mathbf{z}^{[l]}\big)$$

1. Per ogni j,

$$\frac{\partial a_j^{[L]}}{\partial z_j^{[L]}} = \sigma'\big(z_j^{[L]}\big), \qquad \frac{\partial \mathcal{L}}{\partial a_j^{[L]}} = a_j^{[L]} - y_j.$$

   Quindi

$$\delta_j^{[L]} = \frac{\partial \mathcal{L}}{\partial z_j^{[L]}} = \frac{\partial \mathcal{L}}{\partial a_j^{[L]}} \frac{\partial a_j^{[L]}}{\partial z_j^{[L]}} = \big(a_j^{[L]} - y_j\big)\, \sigma'\big(z_j^{[L]}\big),$$

   ossia, in forma vettoriale,

$$\delta^{[L]} = \sigma'\big(\mathbf{z}^{[L]}\big) \circ \big(\mathbf{a}^{[L]} - \mathbf{y}\big).$$

2. Per ogni j, k,

$$z_k^{[l+1]} = \sum_{s=1}^{n_l} w_{ks}^{[l+1]}\, \sigma\big(z_s^{[l]}\big) + b_k^{[l+1]} \implies \frac{\partial z_k^{[l+1]}}{\partial z_j^{[l]}} = w_{kj}^{[l+1]}\, \sigma'\big(z_j^{[l]}\big).$$

   Dato che

$$\delta_j^{[l]} = \frac{\partial \mathcal{L}}{\partial z_j^{[l]}} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{[l+1]}} \frac{\partial z_k^{[l+1]}}{\partial z_j^{[l]}} = \sum_{k=1}^{n_{l+1}} \delta_k^{[l+1]} \frac{\partial z_k^{[l+1]}}{\partial z_j^{[l]}},$$

   Si ha

$$\delta_j^{[l]} = \sum_{k=1}^{n_{l+1}} \delta_k^{[l+1]}\, w_{kj}^{[l+1]}\, \sigma'\big(z_j^{[l]}\big) = \sigma'\big(z_j^{[l]}\big) \sum_{k=1}^{n_{l+1}} w_{kj}^{[l+1]}\, \delta_k^{[l+1]}.$$

   In forma vettoriale,

$$\delta^{[l]} = \sigma'\big(\mathbf{z}^{[l]}\big) \circ \big((W^{[l+1]})^\top \delta^{[l+1]}\big).$$

3. Dato che $z_j^{[l]} = \sum_{s=1}^{n_{l-1}} w_{js}^{[l]}\, a_s^{[l-1]} + b_j^{[l]}$, si ha $\frac{\partial z_j^{[l]}}{\partial b_j^{[l]}} = 1$. Di conseguenza

$$\frac{\partial \mathcal{L}}{\partial b_j^{[l]}} = \frac{\partial \mathcal{L}}{\partial z_j^{[l]}} \frac{\partial z_j^{[l]}}{\partial b_j^{[l]}} = \delta_j^{[l]} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}.$$

4. Dalla stessa espressione,

$$\frac{\partial z_j^{[l]}}{\partial w_{jk}^{[l]}} = a_k^{[l-1]}, \qquad \frac{\partial z_s^{[l]}}{\partial w_{jk}^{[l]}} = 0 \ (s \neq j),$$

   e dunque

$$\frac{\partial \mathcal{L}}{\partial w_{jk}^{[l]}} = \sum_{s=1}^{n_l} \frac{\partial \mathcal{L}}{\partial z_s^{[l]}} \frac{\partial z_s^{[l]}}{\partial w_{jk}^{[l]}} = \delta_j^{[l]}\, a_k^{[l-1]}.$$

■